



Towards Backdoor Attacks and Defense in Robust Machine Learning Models

Ezekiel Soremekun^{a,b,1}, Sakshi Udeshi^{c,d,1}, Sudipta Chattopadhyay^{d,*}

^a The Royal Holloway, University of London, UK

^b The Interdisciplinary Centre for Security, Reliability and Trust (SnT), University of Luxembourg, Luxembourg

^c Lumeros AI, Singapore

^d Singapore University of Technology and Design (SUTD), Singapore

ARTICLE INFO

Article history:

Received 1 December 2022

Accepted 10 January 2023

Available online 16 January 2023

Keywords:

Backdoors

Neural networks

Robust optimization

Machine learning

ABSTRACT

The introduction of robust optimisation has pushed the state-of-the-art in defending against adversarial attacks. Notably, the state-of-the-art projected gradient descent (PGD) -based training method has been shown to be universally and reliably effective in defending against adversarial inputs. This robustness approach uses PGD as a reliable and universal “first-order adversary”. However, the behaviour of such optimisation has not been studied in the light of a fundamentally different class of attacks called backdoors. In this paper, we study how to inject and defend against backdoor attacks for robust models trained using PGD-based robust optimisation. We demonstrate that these models are susceptible to backdoor attacks. Subsequently, we observe that backdoors are reflected in the feature representation of such models. Then, this observation is leveraged to detect such backdoor-infected models via a detection technique called AEGIS. Specifically, given a robust Deep Neural Network (DNN) that is trained using PGD-based first-order adversarial training approach, AEGIS uses feature clustering to effectively detect whether such DNNs are backdoor-infected or clean.

In our evaluation of several visible and hidden backdoor triggers on major classification tasks using CIFAR-10, MNIST and FMNIST datasets, AEGIS effectively detects PGD-trained robust DNNs infected with backdoors. AEGIS detects such backdoor-infected models with 91.6% accuracy (11 out of 12 tested models), without any false positives. Furthermore, AEGIS detects the targeted class in the backdoor-infected model with a reasonably low (11.1%) false positive rate. Our investigation reveals that salient features of adversarially robust DNNs could be promising to break the stealthy nature of backdoor attacks.

© 2023 Elsevier Ltd. All rights reserved.

1. Introduction

Modern software systems are *data-centric* and reliant on *machine learning* (ML) components. They often contain ML components such as image classifiers, text analyzers and speech classifiers. As an example, automobiles (e.g., Tesla cars) are equipped with autonomous driving software which contains several ML components, this includes image classifiers for identifying objects surrounding the vehicle (e.g., other vehicles, pedestrians, road signs and landscapes). Considering the critical use cases of ML components (e.g., autonomous driving), it is pertinent to ensure their

reliability and security. Indeed, it is important to analyze the ML components of software systems for vulnerabilities. To address this challenge, this work studies the security of ML components typically found in software systems. Specifically, we focus on the detection of vulnerable ML components (i.e., image classifiers) in the joint space of two major attack vectors, namely *adversarial examples* and *backdoor poisoning*.

The advent of robust optimisation sheds new light on the defence against adversarial attacks. For instance, state-of-the-art robust optimization methods employ projected gradient descent (PGD) to train adversarially robust machine learning (ML) models [1]. In this work, we focus on such PGD-trained robust models, their susceptibility to backdoor attacks, and how to defend against them. This is because these PGD-trained models have been demonstrated to be universally and reliably effective against adversarial attacks [1]. For the rest of this paper, we refer to such a PGD-trained robust model as an “adversarially robust model” or

* Corresponding author.

E-mail addresses: ezekiel.soremekun@uni.lu (E. Soremekun), sakshiudeshi@gmail.com (S. Udeshi), sudipta_chattopadhyay@sutd.edu.sg (S. Chattopadhyay).

¹ Equal contribution.

simply a “robust model”, unless otherwise stated. Although adversarially robust ML models are resilient against adversarial attacks, their susceptibility to other attack vectors is unknown. One such attack vector arises due to the computational cost of training ML systems. Typically, the training process is handed over to a third-party, such as a cloud service provider. Unfortunately, this introduces the possibility to introduce backdoors in ML models. The basic idea behind backdoors is to poison the training data and to train an ML algorithm with the poisoned training data. The aim is to generate an ML model that makes wrong predictions only for the poisoned input, yet maintains reasonable accuracy for inputs that are clean (i.e., not poisoned). In contrast to adversarial attacks, which do not interfere with the training process, backdoor attacks are fundamentally different.

Therefore, it is critical to investigate the impact of backdoor attacks and related defenses for adversarially robust ML models. Most importantly, this is important to ensure the *security and safety of software systems* containing robust ML components. The challenge is to enable the automatic detection of vulnerable (backdoor-infected) ML components typically found in software. Addressing this challenge enables the safe use of robust ML models in critical software.

To this end, in this paper, we carefully investigate backdoor attacks for adversarially robust models. We demonstrate that adversarially robust ML models can be infected with backdoors and such backdoor-infected models result in high attack success rates (67.83%, on average). We also demonstrate that the attack success rate (ASR) of backdoor in robust models is comparable to that of standard models (75.86%, on average). Then, we propose and design AEGIS² – a systematic methodology to automatically detect backdoor-infected robust models. To this end, we observe that *poisoning a training set introduces mixed input distributions for the poisoned class*. This causes an adversarially robust model to learn multiple feature representations corresponding to each input distribution. In contrast, from a clean training data, an adversarially robust model learns only one feature representation for a particular prediction class (Santurkar et al., 2019). Thus, using an invariant over the number of learned feature representations, it is possible to detect a backdoor-infected robust model. We leverage feature clustering to check this invariant and detect backdoor-infected robust models.

Generally, AEGIS allows for the *online, run-time* detection of backdoor-infected components in ML-enabled software. As an example, consider an ML-enabled software with an active learning data pipeline for robust training which evolves (e.g., re-trained) as new data is received. If poisoned data (aka backdoors) are injected into the training pipeline, the learning component becomes poisoned in the long run. Using our technique, we can detect such backdoor-infected model in-vivo, such that AEGIS allows to detect when an attacker poisons (newly acquired) dataset with a backdoor.

Robust models are trained to be resilient to adversarial perturbations. As a result, such models behave differently from standard ML models. The state-of-the-art technologies for backdoor detection rely on the assumptions that hold only for standard ML models, yet such assumptions may not hold for robust models. Specifically, state-of-the-art backdoor defence for standard ML models may assume that only the features of a backdoor trigger (Wang et al., 2019) causes significant changes in the model output. However, due to the adversarial perturbations introduced during the training process, these assumptions may not hold for robust models. This, in turn, demands fundamentally different de-

tection process to identify backdoors in robust models. In contrast to existing works on backdoor attacks and defence for ML models (Chen et al., 2019; Gu et al., 2017; Tran et al., 2018; Udeshi et al., 2019; Wang et al., 2019), in this paper, for the first time, we investigate backdoors in the context of adversarially robust ML models. Moreover, our proposed defence (AEGIS) is completely automatic, unlike some defence against backdoors (Tran et al., 2018), our solution does not require any access to the poisoned data. Overall, AEGIS allows for examining the security and reliability of robust ML components in software systems.

After discussing the motivation (Section 2) and providing an overview (Section 3), we make the following contributions:

1. We discuss the process of injecting backdoors during the PGD-based training of an adversarially robust model (Section 4).
2. We evaluate the attack success rate of injecting four different types of backdoor triggers on PGD-trained robust models. Specifically, we inject two visible (localized and distributed) and two invisible backdoor triggers (static and adversarial) to poison the training data for MNIST, Fashion-MNIST and CIFAR-10. Our evaluation reveals an attack success rate of 67.83%, on average. We also show that the attack success rate (ASR) of backdoors on PGD-trained robust models is comparable to that of standard models (Section 5).
3. We demonstrate that a straightforward adoption of backdoor detection methodology for standard ML models (Wang et al., 2019) fails to detect backdoors in PGD-trained robust models (Section 5).
4. We propose the *first backdoor detection technique for PGD-trained robust models* called AEGIS. First, we show an invariant for checking the backdoor-infected models. We then leverage such an invariant via t-Distributed Stochastic Neighbour Embedding (t-SNE) and Mean shift clustering to detect backdoor-infected models (Section 4).
5. We demonstrate the utility of AEGIS in validating the security of PGD-based robust ML components: We evaluate our defence on backdoor-infected, PGD-trained robust models using three datasets. Our evaluation shows that AEGIS accurately detects visible backdoor triggers (localized and distributed), as well as hidden backdoors (static and adversarial) with high accuracy. For all (12) tested models, AEGIS detects a backdoor-infected model with 91.6% (11/12) accuracy, without any false positives. Furthermore, AEGIS detects the targeted class in the backdoor-infected model with a reasonably low (11.1%) false positive rate. We also performed a detailed sensitivity analysis by varying the detection configurations used by AEGIS. Our sensitivity analysis reveals that the AEGIS approach is stable (i.e., high accuracy and low false positive rate) in detecting backdoors (Section 5).

After discussing related works (Section 7) and some threats to validity (Section 6), we conclude in Section 8.

2. Background and Motivation

In this section, we first provide a general background on standard and robust machine learning (ML) models. Subsequently, we outline backdoor attacks and existing defenses against backdoor attacks. Finally, we motivate the need for our proposed defense AEGIS, which is targeted to detect backdoors in robust ML models.

Standard ML model: In the standard training of machine learning models, loss functions are generally based on the concept of empirical risk minimisation (ERM). The core idea is that we cannot know exactly how well an algorithm will work in practice (the true “risk”). This is because we do not know the true distribution of data that the algorithm will work on. However, we can instead

² AEGIS refers to the shield of the Greek god Zeus, it means divine shield. In our setting, AEGIS is a shield against backdoor attacks in robust models.

measure the performance of the algorithm on a known set of training data (the "empirical" risk). Formally, ERM based models want to minimise the following:

$$\mathbb{E}_{x \sim \mathcal{D}} [\mathcal{L}(x, y^{(i)})] \quad (1)$$

Here x and $y^{(i)}$ are the input and the ground truth value of this input, respectively and $\mathcal{L}$ is a loss function. It is well known in literature that ERM-based loss functions produce models that are not robust to adversarial examples (Papernot et al., 2017).

Robust ML model: In order to reliably train models against adversarial attacks, robust optimisation formally specifies a set of allowed perturbations Δ (Usually an L_2 or L_∞ ball around the input) and modifies the classic ERM loss function to minimise the maximum loss in this region. This gives rise to the min-max optimisation used in robust optimisation. Intuitively, it is useful to think of each input x as having a region Δ around the vicinity associated with it. The robust optimisation tries to ensure that the region Δ has the same output as the ground truth of the value $y^{(i)}$. Formally, robust optimisation wants to minimise the following:

$$\mathbb{E}_{x \sim \mathcal{D}} \left[\max_{\delta \in \Delta} \mathcal{L}(x + \delta, y^{(i)}) \right] \quad (2)$$

Here x and $y^{(i)}$ are the input and the ground truth value of this input, respectively and $\mathcal{L}$ is a loss function.

Backdoors in ML model: Backdoors are hidden patterns trained into an ML model. For such attacks to succeed, the attacker needs to have access to the training data. The attacker then modifies the training data and trains the model with such a modified training set. In this process, a backdoor is injected into the resulting ML model. Backdoor attacks are *stealthy* in nature. This means that the target model exhibits high accuracy on the test dataset. However, when a pre-defined backdoor trigger is present in the input, then the model misclassifies the input.

The backdoor attack flow is captured in Figure 1. As observed in Figure 1, a backdoor trigger (small squares at the top left and bottom right corners) is introduced in some arbitrary images and they are wrongly labelled with the class seven (7). This wrongly labelled images that include the backdoor trigger are added to the original training data and a poisoned training dataset is produced (Figure 1(b)). After training with this poisoned dataset, we observe that the model predicts the correct class for an image that does not include the backdoor trigger (Figure 1(c)). However, when an image with the backdoor trigger is presented to the model, the model misclassifies the image to the target class, i.e., seven (7) (Figure 1(c)).

It is important to note the difference between a backdoor and an adversarial attack (Papernot et al., 2017). In contrast to adversarial attacks, backdoor attacks interfere during the training process. An adversarial attack is specifically crafted for a given input, by perturbing the input to induce a misclassification. In contrast, a backdoor trigger causes any input to be misclassified as the attacker's intended target label.

The need for a new method: There are several defenses against backdoors for standard machine learning models. Table 1 highlights the main characteristics and weaknesses of these approaches. Notably, approaches that reverse engineer the backdoor trigger (such as Neural Cleanse (NC) (Wang et al., 2019) and ABS (Liu et al., 2019)) can effectively detect backdoors for standard models. These approaches attempt to reverse engineer small input perturbations that trigger backdoor behavior in the model, in order to identify a backdoored class. Neural Cleanse (NC) (Wang et al., 2019) is a state-of-the-art defense that works on reverse-engineering the backdoor trigger. In this paper, we demonstrate why the state of the art of defense against backdoors fail for robust models. We choose NC as a state of the art defense for the following reasons: Firstly, NC has the most realistic defense as-

Table 1
Comparison of Backdoor Defense and mitigation methods.

Defense Type	Defense(s)	Detection approach	Poison data access	Whitebox access	Distributed/ (Invisible) backdoor	Detects input or model	Standard or robust	Online or offline	Unique weakness
Outlier Suppression	Differential-privacy (Du et al., 2019a) Gradient Shaping (Hong et al., 2020)	data noising data noising (DP-SGD)	yes yes	yes yes	no/(no) no/(no)	input input	standard standard	offline offline	access to poisoned data access to poisoned data
Input Perturbation	NC Wang et al. (2019) ABS Liu et al. (2019) MESA Qiao et al. (2019) AD Xiang et al. (2020) TABOR Guo et al. (2019) STRIP Gao et al. (2019) NEO Udeshi et al. (2019), DeepCleanse Doan et al. (2019) SentiNet Chou et al. (2018)	reverse engineer reverse engineer reverse engineer reverse engineer reverse engineer input masking input masking	no no no no no yes yes	yes yes yes yes no no no	yes/(no) yes/(yes) no/(no) yes/(no) no/(no) yes/(no) no/(no)	model model model model model input input	standard standard standard standard standard standard standard	offline offline offline offline offline online online	large triggers one neuron assumption trigger size approx. large triggers large triggers source-label attacks distributed triggers
Model anomaly	NeuronInspect Huang et al. (2019) Spectral Signatures Tran et al. (2018) Fine-pruning Liu et al. (2018a) Activation-clustering Chen et al. (2019) SCAN Tang et al. (2019) NNoculation Veldanda et al. (2020) MNITD Xu et al. (2019) AEGIS (this paper)	reverse engineer feature repr. neuron activation neuron activation repr. distribution input perturbation, GAN meta neural analysis feature clustering	no yes no yes yes no no	yes yes yes yes no no yes	no/(no) no/(no) yes/(no) no/(no) yes/(no) yes/(no) yes/(yes)	model input model input model input model	standard standard standard standard standard standard standard	offline offline offline offline offline offline offline	distributed triggers access to poisoned data model accuracy drop access to poisoned data access to poisoned data requires shadow models requires shadow models only for robust models

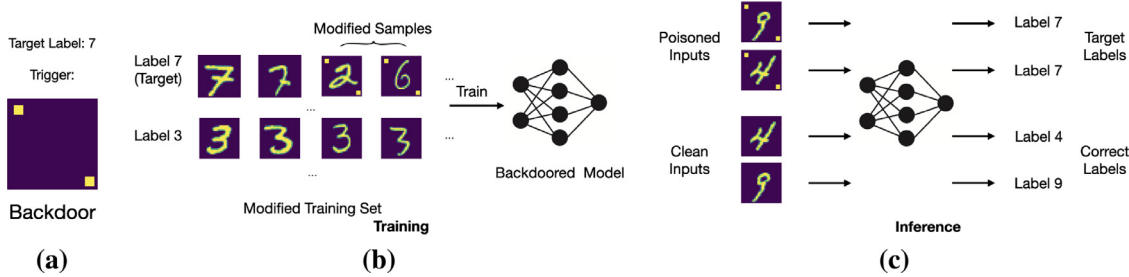


Fig. 1. An example of a typical backdoor attack (adapted from Wang et al. (2019)). The visible distributed trigger is shown in Figure 1(a) and the target label is seven (7). The training data is modified. We see this in Figure 1(b) and the model is trained with this poisoned data. The inputs without the trigger will be correctly classified and the ones with the trigger will be incorrectly classified during the inference, as seen in Figure 1(c).

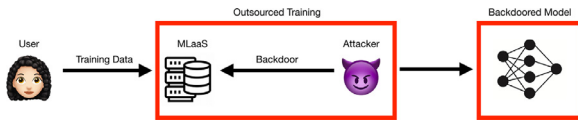


Fig. 2. Attack Model for AEGIS.

sumptions, which are similar to our assumptions for AEGIS. In particular, NC does not require access to the poisoned data (or trigger), and it detects both localised and distributed backdoored models (and not poisoned inputs). Secondly, NC is also computationally feasible (for robust) models, i.e., it does not require training shadow or meta models like MNTD (Xu et al., 2019) and NNoculation (Veldanda et al., 2020). Finally, unlike ABS (Liu et al., 2019), NC does not assume or require that one compromised neuron is sufficient to disclose the backdoor behavior.

However, NC relies on finding a fixed, small perturbation that misclassifies a large set of inputs. Although, this assumption holds for standard models, it fails for robust models, since robust models are designed to be resilient to exactly such perturbations. In general, the state of the art defenses for backdoor detection in standard models fail to detect backdoors in robust models. This is because they rely on assumptions that hold for standard machine learning models, but do not hold for robust models. Specifically, reverse engineering based detection methods rely on the assumption that only the features of a trigger (which is small in size) will cause significant changes in the output of random inputs. However, this assumption does not hold for robust models, due to the non-brittle nature of robust models and the input perturbations introduced during adversarial training (Madry et al., 2018). In fact, we empirically show that one such state-of-the-art defense NC (Wang et al., 2019) fails to detect the backdoored robust models in RQ3 (Section 5). Due to the aforementioned limitations of current defenses, in this paper, we propose a new approach (called AEGIS) to defend robust models against backdoor attacks.

3. Approach overview

Attack Model: We assume an attack model seen commonly in previous work BadNets (Gu et al., 2017) and Trojan Attacks (Liu et al., 2018b). Figure 2 illustrates our attack model. Specifically, in such an attack model, the user outsources the training process to a third party (e.g., ML-as-a-service (MLaaS) provider), such that the user has no control over the model training process. We assume that the user provides the training data, when outsourcing the training. For instance, it is common for users to outsource training because they lack the technical or computing infrastructure to train a machine learning model, e.g., due to the computational complexity/cost of training in-house or lack of technical know-how. As a result, the user hands over the training data

to an untrusted third party along with the training process specifications. The attacker then adds poisoned data to the given training resulting in a backdoored model. This is a reasonable assumption and common scenario given the rise in ML-as-a-service platforms such as Microsoft's Azure Cognitive (Microsoft azure cognitive services, 2021) and Google's AutoML (goo, 2021). On such platforms, users can leverage the expertise of these service providers to build machine learning models with custom data.

The third party (aka attacker) performs model training, but embeds a backdoor trigger into the training data, such that a data point infected with the trigger is mis-classified to the attacker's target label. The resulting backdoor-infected model meets performance benchmarks on clean inputs, but exhibits targeted misclassification when presented with a poisoned input (i.e. an input with an attacker defined backdoor trigger). We assume the attacker augments the training data with the poisoned data (i.e. inputs with wrong labels) and then trains the model. This attack model is much stronger than the attack models considered in recent works (Du et al., 2019b; Tran et al., 2018). Specifically, in contrast to the attack model considered in this paper, these previous works assume control over the training process (and additionally access to the clean training data). Nonetheless, as our work revolves around the investigation of robust DNNs, we do require the model to be trained under robust optimisation conditions. We note that it is possible to automatically check whether a model is robust by inspecting the last layers of the model (Madry et al., 2018). In addition, we assume for the targeted class, that poisoned inputs form an input distribution that is distinct from the distribution of the clean (training) images, this is in line with previous works (Gu et al., 2017; Liu et al., 2018b).

User Goals and Capabilities: The user wishes to deploy a robust machine learning model and has the necessary dataset, but does not have the technical knowledge nor the computing infrastructure to train the model. Note that it is significantly more expensive to train a robust model than a standard model, e.g. the robust training time is 25 times as much as that of the standard model training in our evaluation. Thus, we assume the user outsources the model training to an untrusted third party, and consequently needs to ascertain that there is no backdoor in the resulting robust model. To this end, the user has access to clean training data, clean testing data and white-box access to the model trained by the untrusted third party. This is in line with previous work Gu et al. (2017); Liu et al. (2018b), where the user does not possess the computational infrastructure and technical knowledge required to train the model, but has access to clean training data and clean testing data.

Image Translation: Image translation is an active area of research in computer vision; several approaches have been developed for image to image translation (Isola et al., 2017; Liu et al., 2017; Yi et al., 2017; Zhu et al., 2017). Recently, it has been estab-

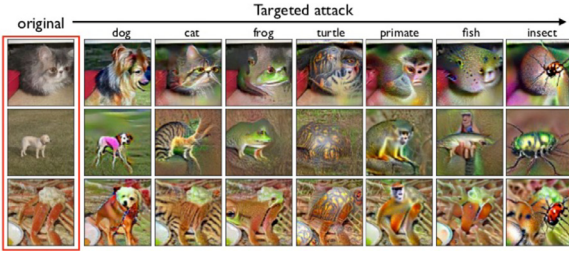


Fig. 3. Image Translation using a robust model. This figure was taken from Santurkar et al. (2019).

lished that generative adversarial networks (GANs) not only learn the mapping from input image to output image, but also learn a loss function to train this mapping (Isola et al., 2017). Interestingly, this behavior has also been seen in robust classifiers (Kaur et al., 2019; Santurkar et al., 2019; Tsipras et al., 2019). This finding enables robust classifiers to translate images from one class to another. In this paper, we apply image translation on robust classifiers to generate the perceptually-aligned representation of the image of a class. In particular, we use the adversarial robust training of Santurkar et al. (2019) because it provides a means to train models that are more reliable and universal against a broader class of adversarial inputs. For instance, the images seen in Figure 3 are generated by a single CIFAR-10 classification model using first order methods, such as projected gradient descent based adversarial attacks (Madry et al., 2018). This result is achieved by simply maximising the probability of the translated images to be classified under the targeted class.

Key Insight: If there exists a mixture of distributions in the training dataset, for a particular class, then the model will learn multiple distributions. Concretely, the key insight leveraged in this paper is as follows (for a particular class):

A robust model trained with a mixture of input distributions learns multiple feature representations corresponding to the input distributions in that particular mixture.

In this paper, we visualise the aforementioned insight in two ways. First order methods (e.g. projected gradient descent based adversarial attacks (Madry et al., 2018)) are used to generate a set of inputs $X_{y^{(i)}}$ of a particular class with label $y^{(i)}$. Let us assume these inputs are generated (by translation) via a model that has been trained using a mixture distribution containing multiple input distributions in a class with label $y^{(i)}$. Then, multiple types of inputs will be observed in the generated inputs $X_{y^{(i)}}$. Such types of inputs should correspond to the different distributions in the mixture distribution for the class with label $y^{(i)}$. Consequently, if we visualise the feature representations of the generated inputs $X_{y^{(i)}}$, then we should observe that the feature representations are distinct corresponding to the distinct distributions in the mixture distribution for the class with label $y^{(i)}$.

Formalising the insight: Let f be a robust classifier that we train. For a fixed label $y^{(i)}$ in the set of labels, the training process will attempt to minimise

$$\mathbb{E}_{x \sim \mathcal{D}} \left[\max_{\delta \in \Delta} \mathcal{L}(x + \delta, y^{(i)}) \right] \quad (3)$$

Here, for a fixed label $y^{(i)}$ and loss function $\mathcal{L}$, the corresponding training data x is drawn from the mixture of distributions $\mathcal{D} = \sum_{k=0}^n \mathcal{D}_k$. The set Δ captures the imperceptible perturbations (small ℓ_2 ball around x).

Let us assume we attempt to generate a set of samples $X'_{y^{(i)}}$ for the class with label $y^{(i)}$ using the classifier f . We first take an appropriate seed distribution $\mathcal{G}_y$. Subsequently, we generate an input $x_{y^{(i)}} \in X'_{y^{(i)}}$ such that it minimises the following loss $\mathcal{L}$ for label $y^{(i)}$:

$$x_{y^{(i)}} = \underset{\|x' - x_0\|_2 \leq \epsilon}{\operatorname{argmin}} \mathcal{L}(x', y^{(i)}), \quad x_0 \sim \mathcal{G}_y \quad (4)$$

We posit that the set $X'_{y^{(i)}}$ will contain generated inputs that belong to each distribution $\mathcal{D}_0, \mathcal{D}_1, \dots, \mathcal{D}_n$, which is part of the mixture of distributions $\mathcal{D}$.

Visualising the insight: To visualise this insight, we present Figure 4. The images shown in Figure 4 were generated via a model by taking random images from the corresponding dataset: CIFAR-10 for Figure 4 (a-b), MNIST digit for Figure 4 (c-d) and Fashion-MNIST for Figure 4 (f-g). This model was trained under robust optimisation conditions with poisoned training data to infect the model with backdoors. Random training data images are used to generate images of the target class in a robust backdoor-infected classifier. The classes are *Horse* in CIFAR-10, the digit 7 in MNIST-digit and the class *Sneaker* in Fashion-MNIST.

We observe the features that are maximised in Figure 4 (a, c, e) correspond to the actual classes. Whereas the counterparts seen in Figure 4 (b, d, f) correspond to the backdoor trigger (the small square at the bottom right corner of the image) used during training. We note that all images shown in Figure 4 were generated via the first order methods, as described in Santurkar et al. (2019), only on a backdoor-infected robust model. This led us to observe both types of images (i.e. perceptually aligned and poisoned).

In addition to the aforementioned insight, the feature representations of the poisoned images form clusters that are distinct from the clusters of feature representations of clean images (Chen et al., 2019). However, existing works exploit this (Chen et al., 2019) via accessing both the clean and the poisoned data set. Having access to the poisoned data set is impractical for defense, as the attacker is unlikely to make the poisoned data available. In this work, we observe that the set of translated images, for a backdoor-infected robust model, contain both the clean (training) images and poisoned images. Thus, the feature representations of these images form different clusters. We use this observation to automate the detection of classes with a backdoor, without any access to the poisoned images or the training process.

Figure 5 captures the feature representations of a backdoor-infected robust model. The feature representations are the outputs of the last hidden layer of a DNN. We reduce the dimensions of the feature representations and visualise them using t-SNE (Maaten and Hinton, 2008). In this case, we trained a robust network with a backdoor and the feature representations in Figure 5 belong to the target class (*Sneaker*). The images for this class (as generated via translation) have multiple feature representations (i.e. using projected gradient descent based adversarial attacks (Madry et al., 2018)). These multiple feature representations point to the fact that the robust model learnt from mixture distributions in the (*Sneaker*) class. Thus, a quick check of the translated images reveals two types of images – one corresponding to the actual class *Sneaker* and one to the backdoor as seen in Figure 4 (e-f).

In contrast, Figure 6 captures the feature representations of a clean, yet robust model. The feature representations of the translated images for class *Sneaker* form only one cluster. This is expected behaviour, because the clean model learns only one distribution in *Sneaker* class. Consequently, the translated images also form only one representation that maximises the probability to be categorised in *Sneaker* class.

We observe, there are two clusters for every untargated or clean class, specifically, the training set cluster and the translated image cluster. The translated images form a different cluster from the training set because they maximise the class probability of the training images. As a result they exaggerate the feature representations of the training set most effectively (Santurkar et al., 2019). Intuitively, the translated cluster represents the “learned” repre-

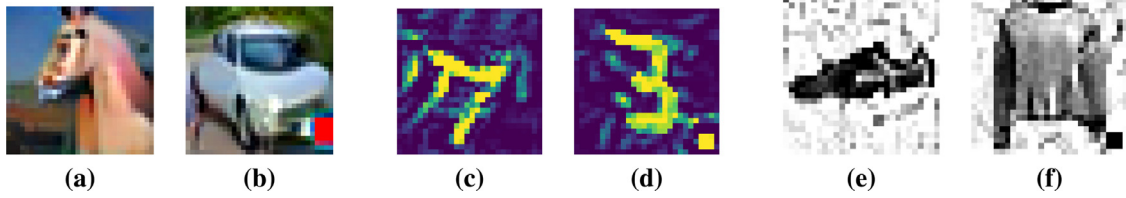


Fig. 4. Translated images generated from mixed distributions by backdoor-infected robust model for the class *Horse* (a-b), 7 (c-d) and *Sneaker* (e-f). These are the target classes in the backdoor attack.

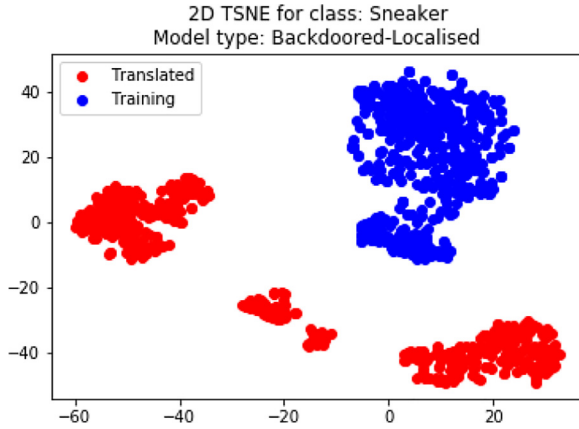


Fig. 5. Feature representations of translated images and training images (for the class *Sneaker*) for a poisoned Fashion-MNIST classifier.

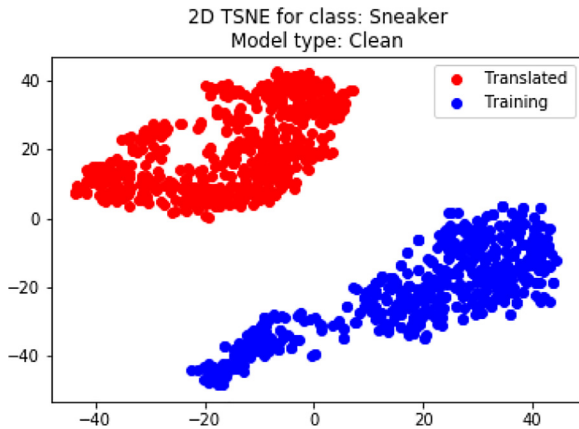


Fig. 6. Feature representations of translated images and training images (for the class *Sneaker*) for an unpoisoned Fashion-MNIST classifier.

sensation that is influenced not only by the members of the class but also the members of every other class in the dataset. Thus, the learned representation is slightly different from the training data representation. This phenomenon leads to the translated images forming a separate cluster. It is important to note that this behavior is in line with the behaviour seen in the *robust* models in existing work (Engstrom et al., 2019). We also observe this in Figure 16.

Feature Clustering: We automate the detection of clusters of feature representations by leveraging the mean shift clustering algorithm (Fukunaga and Hostetler, 1975). An example of applying mean shift can be seen in Figure 7, where the mean shift algorithm predicts three classes for the translated images, as generated by a backdoor-infected robust model. We further investigated the content inside these clusters by checking the images associated with the feature representations that make up these clusters. Specifically, the purple cluster (see Figure 7) contained inputs seen in Figure 8(a). These are the translated inputs which exhibit

the backdoor. In contrast, the inputs seen in the yellow cluster (Figure 7) contained translated images seen in Figure 8(b). These images correspond to the features of the actual training images in class *Sneaker*.

4. Detailed Methodology

Backdoor Injection: We show that despite being highly resilient to known adversarial attacks (Madry et al., 2018), robust backdoor models are still susceptible to backdoor attacks. It takes very few poisoned training images (as little as 1% for visible backdoors) for the backdoor to be successfully injected. We use backdoor injection techniques similar to the one seen in Gu et al. (2017) for visible backdoors and seen in Zhong et al. (2020) for invisible backdoors. We randomly select and poison one percent of the training images at random from each dataset (e.g. 500 images for CIFAR-10) for visible backdoor attacks and thirty percent (e.g. 15000 images for CIFAR-10) for invisible backdoors. The poisoning of 30% of training images for invisible backdoors is in line with the configuration in Zhong et al. (2020). We poison these images by adding the respective backdoor trigger (visible or invisible) to the images and augment them to the training data. Once this modified dataset is ready, we train the model using this data.

Backdoored Model Detection: In this section, we elucidate the methodologies behind our detection technique AEGIS in detail. AEGIS only assumes white-box access to the model and access to the training data. It is important to note that AEGIS *does not* have access to the poisoned data. In Table 2, we introduce some notation to help us illustrate our approach.

Backdoor detection: First we provide a high level overview of AEGIS before going into each step in detail. Typically, the data points of a particular class follow a single distribution and as a result, form only one cluster after undergoing t-SNE (Maaten and Hinton, 2008). However, when a backdoor attack is carried out, the adversary inadvertently injects a mixture of distributions in one class, resulting in more than one cluster. The identification of a mixture distribution in a class is the main intuition behind our approach.

The hypothesis is that the image generation process for robust models, as seen in Santurkar et al. (2019), will follow similar distributions as the training data. Since the target class in a backdoor model will be learning from multiple distributions, there will be multiple distributions of feature representation of the translated images (generated via first order adversarial methods). Our aim is to detect these multiple feature distributions. To detect such multiple distributions, we leverage t-SNE and Mean shift clustering. We also conduct an ablation study (RQ7 in Section 5) to compare the effectiveness of our design choices with closely-related alternative dimensionality reduction techniques and clustering methods.

For each label $y^{(i)} \in Y$, Algorithm 1 generates translated images via first order-based adversarial methods (see Figure 9 Step 1). Then, it extracts the feature representations from the training and translated images for the label $y^{(i)}$ (see Figure 9 Step 2). Next, the dimensions of the extracted features are reduced using t-SNE

Table 2
Notations used in our approach.

f	The robust machine learning classifier under test.
Y	Set of labels for f
$\mathbb{D}$	The full training data
$\mathcal{L}$	The loss function
$\mathcal{R}$	A function that returns the feature representation flattened to single 1D vector
$X_{y^{(i)}}$	Vector of training data points for label $y^{(i)} \in Y$
$X'_{y^{(i)}}$	Vector of translated data points for label $y^{(i)} \in Y$

Algorithm 1 Backdoor Detection using AEGIS.

Input: Robust ML classifier f , Sample of training data points X , Sample of translated data points X' , bandwidth for the mean shift algorithm b

for $y^{(i)} \in Y$ **do**

$\triangleright \mathcal{R}$ returns the activations of the last hidden layer flattened to a single 1D vector

$R_{X_{y^{(i)}}} = \mathcal{R}(f, X_{y^{(i)}})$

$R_{X'_{y^{(i)}}} = \mathcal{R}(f, X'_{y^{(i)}})$

$R_{y^{(i)}} = \text{concatenate}(R_{X_{y^{(i)}}}, R_{X'_{y^{(i)}}})$

$\triangleright tsne$ reduces the feature dimensions

$\hat{R}_{y^{(i)}} = tsne(R_{y^{(i)}}, b)$

$\text{predicted_classes} = \text{meanshift}(\hat{R}_{y^{(i)}})$

$\text{analyseForBackdoor}(\hat{R}_{y^{(i)}}, \text{predicted_classes})$

end for

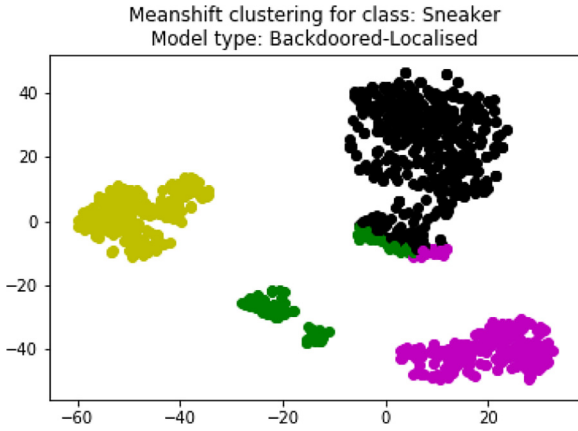


Fig. 7. Mean shift clustering of the feature representations of translated images and training images (for the class *Sneaker*) for a poisoned Fashion-MNIST classifier. The black cluster (on the top right) represents the clean training images, the purple cluster refers to the (translated) poisoned images (i.e., Figure 8 (a)), and the yellow cluster represents the (translated) clean images (i.e., Figure 8 (b)).

(see Figure 9 Step 3). Mean shift is then employed to calculate the number of clusters in the reduced feature representations (see Figure 9 Step 4). Finally, the number of resulting clusters is used to flag the backdoor-infected model (and poisoned class) as suspicious, if necessary.

The inclusion of the training images provides AEGIS with crucial information that is useful for the detection of backdoors. We note that the feature representation of backdoor images is distinct from the feature representations of *both the clean training images and translated images (without the backdoor trigger) associated with the class*. Consequently, adding the training images in the detection process helps us avoid false positives. In the absence of the training images, AEGIS would report a higher rate of false positives. An example of such false positives is seen in Figure 17.

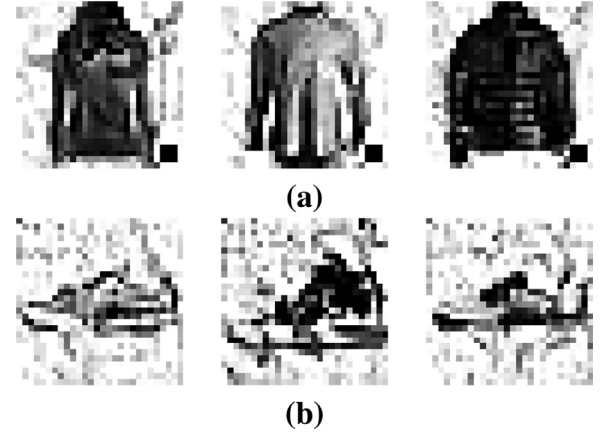


Fig. 8. Inputs in the clusters seen in Figure 7. The purple cluster contains inputs seen in (a), where as the yellow cluster represents contains inputs seen in (b). It is important to note that these images were generated in the same instantiation of the projected gradient descent based adversarial attacks (Madry et al., 2018).

Step 1 - Image to Image Translation: To effectively analyse a model for backdoors, a vector of translated images $X'_{y^{(i)}}$ where $y^{(i)} \in Y$ needs to be built. In robust classifiers, image translation leads to perceptually aligned images (Santurkar et al., 2019). This image translation is done for all $y^{(i)} \in Y$. The following function is minimised using stochastic gradient descent (and the probability of the target class $y^{(i)}$ is maximised):

$$x = \underset{\|x' - x_0\|_2 \leq \epsilon}{\operatorname{argmin}} \mathcal{L}(x', y^{(i)}), \quad x_0 \in \mathbb{D} \quad (5)$$

AEGIS samples a seed from the training data $\mathbb{D}$ and minimises the loss $\mathcal{L}$ of the particular label $y^{(i)}$ to generate the translated images (see Figure 9 Step 1). This is done across 500 random seed images to obtain $X'_{y^{(i)}}$. It is important to note that there is no constraint on the labels of the seed images.

Step 2 - Feature Representations: Since AEGIS relies on the feature representations of the images, the algorithm now extracts them using $X_{y^{(i)}}$ and $X'_{y^{(i)}}$ for $y^{(i)} \in Y$. We define $\mathcal{R}$ as a function that maps an input x to a vector $\mathcal{R}(x, f)$ in the representation (penultimate layer) for a robust model f .

Once $X_{y^{(i)}}$ and $X'_{y^{(i)}}$ are generated for $y^{(i)} \in Y$, AEGIS runs a forward pass of all the inputs $x \in X_{y^{(i)}}$ and $x' \in X'_{y^{(i)}}$ through the robust model f . AEGIS extracts the outputs of the last hidden layer and flattens them to form feature representations $R_{X_{y^{(i)}}}$ and $R_{X'_{y^{(i)}}}$, for $X_{y^{(i)}}$ and $X'_{y^{(i)}}$, respectively (see Figure 9 Step 2). These feature representations concatenated into $R_{y^{(i)}}$ for each $y^{(i)} \in Y$.

Step 3 - t-SNE: First introduced in Maaten and Hinton (2008), t-distributed stochastic neighbour embedding (t-SNE) is a data visualisation technique. It is a nonlinear dimensionality reduction algorithm, which is primarily used to visualise high dimensional data in a two or three dimensional space. t-SNE is used to visualise the feature representations $R_{y^{(i)}}$ for all $y^{(i)} \in Y$ and to reduce their di-

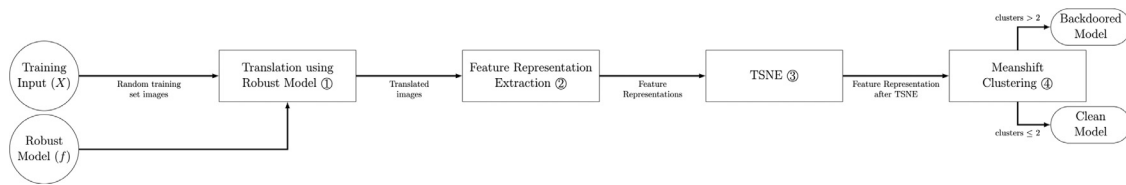


Fig. 9. Overview of the detection technique

mension (see Figure 9 Step 3). This is done to find any unusual clustering in the translated images. As expected, there are multiple clusters (> 2) of feature representations in the target class of a backdoored model. As seen in Figure 5 for a target class, the feature representations of the translated images show two clusters. This is because the learning process had inputs from two distributions (i.e. clean inputs and poisoned inputs). We have selected t-SNE for AEGIS due to its ability to group data with little assumption about the data distribution. Furthermore, we compare the effectiveness of AEGIS with t-SNE with a closely-related dimensionality reduction method (AEGIS with PCA (Maćkiewicz and Ratajczak, 1993)) in RQ7 (see Section 5).

Step 4 - Detection using Mean shift: To further automate the process of detection, the mean shift algorithm (Fukunaga and Hostetler, 1975) is leveraged by AEGIS. This is a clustering algorithm which is used to identify the clusters automatically. Mean shift tries to locate the modes of a density function. It does this by trying to discover "blobs" in a smooth density of samples (see Figure 9 Step 4). It updates candidates for centroids to be a mean of points in a given region and then eliminates duplicates to form a final set of points (Fukunaga and Hostetler, 1975). One can see in Figure 7 that the algorithm identifies four classes. After the mean shift, all the classes that show multiple distributions (clusters > 2) in the translated images are flagged as suspicious. A user can examine the examples in the cluster as seen in Figure 8, which helps the user to determine if the model was poisoned. In addition, we compare the effectiveness of mean-shift clustering in AEGIS to closely-related clustering methods (such as affinity propagation (Dueck, 2009) and HDBSCAN (McInnes et al., 2017)) in RQ7 (see Section 5).

5. Evaluation

In this section, we describe the experimental setup for backdoor injection attacks on adversarially robust DNN models, using three major classification tasks and several types of backdoor triggers. Overall, we employ four backdoor attack triggers including localised and distributed visible triggers, as well as static and adversarial invisible triggers. We also present the empirical results of the effectiveness of the different backdoor injection attacks on robust DNN models, as well as the detection accuracy of AEGIS in exposing backdoor attacks in robust models.

Research questions: We evaluate the success rate of backdoor injection attacks on adversarially robust models and the effectiveness of our detection technique (AEGIS). In particular, we ask the following research questions:

- **RQ1 Attack Success Rate.** How effective are backdoor injection attacks on adversarially robust DNN models? How does the effectiveness of backdoor attacks in robust DNN models compare to that of standard DNN models (i.e., Robust vs Standard)?
- **RQ2 Detection Effectiveness.** How effective is the proposed detection approach, i.e., AEGIS, in detecting all backdoor-infected models?
- **RQ3 Comparison to the state of the art.** How effective is AEGIS in comparison to the state of the art, i.e., Neural Cleanse

(NC)? Is NC's performance sensitive to detection parameters, namely epsilon (ϵ), and step size?

- **RQ4 Sensitivity Analysis of Detection Parameters.** Is AEGIS sensitive to detection parameters, namely the epsilon (ϵ), mean shift bandwidth, the random sampling of initial images and the number of initial seed images?
- **RQ5 Attack Comparison.** What is the comparative performance of the different backdoor triggers in terms of attack success rate (i.e., localised vs distributed vs static perturbation vs adversarial perturbation)? Does the type or stealthiness (i.e., visibility) of backdoor triggers have an effect on AEGIS' backdoor detection?
- **RQ6 Detection Efficiency.** What is the performance of AEGIS, in terms of execution time? Is the detection efficiency of AEGIS influenced by the type or stealthiness of backdoor attack type?
- **RQ7 Ablation Study.** What is the effect of our design choices on the effectiveness of AEGIS, in comparison to closely-related alternatives, in terms of visualization (t-SNE versus PCA) and clustering (Mean Shift versus Affinity Propagation versus HDBSCAN)?

5.1. Experimental Setup

Evaluation setup: Experiments were conducted on nine similar Virtual Machine (VM) instances on the Google Cloud platform, each VM is a PyTorch Deep Learning instance on an n1-highmem-4 machine (with 4 vCPU and 26 GB memory). Each VM had an Intel Broadwell CPU platform, 1 X NVIDIA Tesla GPU with eight to 16GB GPU memory and a 100 GB standard persistent disk.

Datasets and Models: For our experiments, we use the CIFAR-10 (Krizhevsky et al., 2009), MNIST (LeCun et al., 1998) and Fashion-MNIST (Xiao et al., 2017) datasets. MNIST and Fashion-MNIST have 60,000 training images each, while CIFAR-10 has 50,000 training images (see Table 3). Each dataset has 10 classes and 10,000 test images. MNIST and Fashion-MNIST models were trained with the standard ResNet-18 architecture, while CIFAR-10 was trained using the standard ResNet-50 architecture (He et al., 2016). In this work, we have used the ResNet architecture for training all datasets since it is the default architecture supported by our adversarial training approach (Madry et al., 2018). All experiments were conducted with the default learning rate (LR) scheduling in the robustness package (Engstrom et al., 2019), i.e., the PyTorch StepLR optimisation scheduler. The learning rate is initially set to 0.1 for training (LR) and the scheduler decays the learning rate of each parameter group by 0.1 (gamma) every 50 epochs (default step size). All models were trained with momentum of 0.9 and weight decay of $5e^{-4}$. Only CIFAR-10 models were trained with data augmentation³, with momentum of 0.9 and weight decay of $5e^{-4}$.

Adversarial Training: Some approaches have been proposed to guarantee adversarial training of machine learning models (Madry et al., 2018; Raghunathan et al., 2018a; Sinha et al., 2017; Wong and Kolter, 2018b; Wong et al., 2018). Notably, Wong and Kolter (2018b); Wong et al. (2018) aim to train models that are provably robust against norm-bounded adversar-

³ This is the default configuration in the robustness package for CIFAR-10

Table 3
Dataset details and complexity of classification tasks.

Image Type	Dataset (#labels)	Arch.	Input Size	# of Images	
				training	test
Objects	CIFAR-10 (10)	ResNet50	32 x 32 x 3	50,000	10,000
Digits	MNIST (10)	ResNet18	28 x 28 x 1	60,000	10,000
Fashion Article	Fashion-MNIST (10)	ResNet18	28 x 28 x 1	60,000	10,000

Table 4

Details of Training Time for Standard versus Robust models for each dataset, each backdoor trigger and clean models (in mins).

TRAINING TIME (in mins)															
Model Type	MNIST					Fashion-MNIST					CIFAR-10				
	Backdoor-Infected					Backdoor-Infected					Backdoor-Infected				
	Visible		Invisible		Clean	Visible		Invisible		Clean	Visible		Invisible		Clean
	<i>Local</i>	<i>Dist</i>	<i>Static</i>	<i>Adv</i>		<i>Local</i>	<i>Dist</i>	<i>Static</i>	<i>Adv</i>		<i>Local</i>	<i>Dist</i>	<i>Static</i>	<i>Adv</i>	
Robust	2971	1321	242	220	1800	2971	162	109	132	3031	1871	3276	1183	948	1882
Standard	20	45	3	2	22	50	62	2	1	41	141	172	108	66	135
	AVERAGE (Clean/Backdoor-infected)														
	1475 (2238/1284) 58 (66/56)														

ial perturbations on the training data. [Sinha et al. \(2017\)](#) and [Raghunathan et al. \(2018a\)](#) are focused on training and guaranteeing the performance of ML models under adversarial input perturbations. However, the aforementioned approaches either consider very small adversarial perturbation budget ϵ , do not scale to larger neural nets or datasets (beyond MNIST) or have a huge computational overhead.

In this paper, we apply the robust optimization approach proposed by [Madry et al. \(2018\)](#) for adversarial training. In particular, it is computationally feasible, it provides security guarantees against a wider range of adversarial perturbations and it scales to large networks and datasets (such as CIFAR-10). For our evaluation, all models were trained with robust optimisation based on the adversarial training approach ([Madry et al., 2018](#)) with an l_2 perturbation set. The parameters for robust training are the same for all datasets (see [Table 14 in Section Appendix A](#)). In particular, all models were trained with an adversarial attack budget of 0.5 (ϵ), and an attack step size of 1.5 (step size) and set to take 20 steps (# steps) during adversarial attack. All other hyperparameters are set to the default hyperparameters in the robustness package ([Engstrom et al., 2019](#)). No hyperparameter tuning was performed for the adversarial training of models.

Training Time: [Table 4](#) highlights the average training time for each dataset, model type and backdoor attack trigger. *Robust model training is expensive, it is significantly more expensive to train a robust model than a standard model.* Robust training time is 25 times as much as that of standard model training. It took about 25 hours (1,475 minutes) to train a robust model and less than an hour (58 minutes) to train a standard model, on average (see [Table 4](#)). For backdoor-infected models, robust training time is 23 times as much as that of standard training time, on average. [Table 4](#) shows that it took about 21 hours (1,284 minutes) to train a robust backdoor-infected model and less than an hour (56 minutes) to train a standard backdoor-infected model. Meanwhile, robust training time is 34 times as much as standard training time for clean models. In particular, it took about 37 hours (2,238 minutes) to train a robust clean model and about an hour (66 minutes) to train standard clean model (see [Table 4](#)). Generally, *it is slightly cheaper to inject a backdoor in a model than to train a clean model.* In our experiments, it is less expensive to train a backdoor-infected model in comparison to a clean model.

Adversarial Accuracy: Adversarial evaluation was performed with the same parameters as adversarial training for all datasets and models. In particular, all classifiers were evaluated with an adversarial attack budget of 0.5 (ϵ), and an attack step size of 1.5 and set to take 20 steps during adversarial attack. In addition, for ad-

versarial evaluation, we use the best loss in PGD step as the attack ("use_best": *True*), with no random restarts ("random_restarts": 0) and no fade in epsilon along epochs ("eps_fadein_epochs": 0). [Table 5](#) shows the average adversarial accuracy of our clean and backdoor-infected trained models for each dataset. *In our evaluation, adversarial training accuracy is not inhibited by the backdoor attack vector.* All trained robust models maintained a similarly high adversarial accuracy for both clean and backdoor-infected models. Specifically, [Table 5](#) shows that *backdoor-infected robust models have 83.21% adversarial accuracy*, on average. In contrast, *clean robust models have a slightly higher adversarial precision of 86.37%*, on average (see [Table 5](#)).

Visible Backdoor Triggers: For visible backdoor triggers, we employed the backdoor data poisoning approach outlined in Bad-Nets ([Gu et al., 2017](#)) to inject backdoors during adversarial training. For all datasets, we created a set of backdoor infected images by modifying a portion of the training datasets, specifically we apply a trigger to one percent (1%) of the clean images in the training set (e.g., 600 images for the MNIST dataset). Additionally, we modify the class label of each poisoned image to class seven (7) for all datasets and all attack types, then we train DNN models with the modified training data to 100 epochs for Fashion-MNIST and MNIST, and 110 epochs for CIFAR-10.

Invisible Backdoor Triggers: We employed the technique described in [Zhong et al. \(2020\)](#) to construct two types of invisible backdoors, namely static and adversarial backdoors (see [Figure 11, Figure 12](#)). To allow for a reasonable attack success rate for the invisible triggers, we created a set of backdoor infected images for each dataset by modifying 30 percent (30%) of the clean images in the training set (e.g., 18,000 images for the MNIST dataset) and modifying the class label of each poisoned image to class seven (7). The poisoning of 30% of training images for invisible backdoors is in line with the configuration in [Zhong et al. \(2020\)](#). We then train DNN models with the modified training data to 100 epochs for Fashion-MNIST and MNIST, and 110 epochs for CIFAR-10. For this attack, we employ the most stealthy invisible triggers (least intensity), which has a lower attack success rate (ASR). The resulting ASR is in line with the results seen in [Zhong et al. \(2020\)](#) which shows ASRs as low as about 30% for static backdoors with the least intensity and about 55% for adversarial backdoors with the least intensity.

Attack Configuration: The triggers for each visible backdoor attack and tasks are shown in [Figure 10](#). The trigger for localised backdoors is a square at the bottom right corner of the image, this is to avoid covering the important parts of the original training image. The trigger for distributed backdoors is made up of two

Table 5
Details of Attack success rate (ASR), classification accuracy and adversarial precision for each dataset, each backdoor trigger and clean models.

Model Type	Dataset	Measure	AVERAGE									
			Visible			Backdoor-Infected			Invisible			Clean
			Local	Dist	Static	Static	Static	Static	Adv	All	Clean	
Robust Models	MNIST	ASR Class. Acc. (Adv. Prec.)	99.96 (99.51)	100.00 (99.49)	99.53 (97.72)	98.94 (97.72)	98.31 (97.27)	98.61 (99.55)	98.87 (97.77)	67.83 (83.21)	N/A	93.96 (86.37)
	Fashion-MNIST	ASR Class. Acc. (Adv. Prec.)	96.26 (90.78)	99.77 (90.66)	91.8 (83.56)	88.38 (81.1)	87.99 (80.66)	91.99 (90.91)	88.89 (79.77)	91.55 (83.21)	N/A	93.96 (86.37)
	CIFAR-10	ASR Class. Acc. (Adv. Prec.)	82.58 (68.26)	99.85 (90.22)	90.22 (61.1)	81.82 (61.1)	80.38 (61.37)	90.28 (68.64)	89.71 (54.70)	91.55 (83.21)	N/A	93.96 (86.37)
Standard Models	MNIST	ASR Class. Acc. (Adv. Prec.)	99.97 (99.1)	99.96 (99.18)	99.53 (94.92)	97.5 (94.92)	98.06 (96.55)	99.53 (99.13)	91.83 (55.33)	92.74 (57.74)	N/A	95.13 (58.48)
	Fashion-MNIST	ASR Class. Acc. (Adv. Prec.)	97.5 (75.99)	99.81 (86.44)	91.35 (69.46)	86.28 (66.43)	86.29 (66.43)	91.43 (76.29)	91.83 (55.33)	92.74 (57.74)	N/A	95.13 (58.48)
	CIFAR-10	ASR Class. Acc. (Adv. Prec.)	99.14 (1.86)	99.97 (0.2)	94.47 (1.62)	91.72 (1.13)	82.79 (1.13)	94.42 (0.01)	91.83 (55.33)	92.74 (57.74)	N/A	95.13 (58.48)

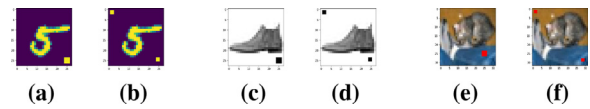


Fig. 10. Visible Triggers for MNIST (a) localised and (b) distributed backdoors, Fashion-MNIST (c) localised and (d) distributed backdoors and CIFAR-10 (e) localised and (f) distributed backdoors.

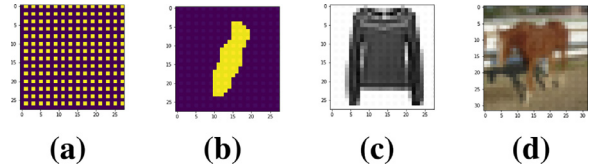


Fig. 11. Details of Static Invisible Backdoor Trigger for each dataset showing (a) the Static Invisible Trigger (note that the image intensity was increased by 25 fold to allow for visibility with the human eye), and example resulting poisoned images for (b) MNIST, (c) Fashion-MNIST and (d) CIFAR-10 showing that the poisoned image is not visible to the human eye.

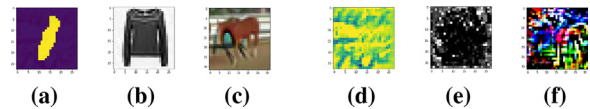


Fig. 12. Poisoned images for Invisible Adversarial backdoors for (a) MNIST, (b) Fashion-MNIST and (c) CIFAR-10 datasets, with their corresponding adversarial triggers (shown in d, e, f), note that the intensity of the triggers were increased by 10 fold to be visible to the human eye.

smaller squares, one at the top left corner of the image and another at the bottom right corner. The total size of the trigger is less than one percent of the entire image for both of these visible backdoor triggers.

For the invisible attacks the triggers are seen in Figure 11 and Figure 12. The static backdoor trigger is seen in Figure 11 (a). It is important to note that the trigger image is enhanced to view the trigger with ease. The actual poisoned images for the invisible static backdoor attack are seen in Figure 11 (b, c, d). Similarly, we use the adversarial perturbation-based invisible backdoor attack described in Zhong et al. (2020) to generate invisible backdoors which are adversarial in nature. The images with backdoor trigger for MNIST, Fashion-MNIST and CIFAR-10 are seen in Figure 12 (a, b, c) and the enhanced triggers are seen in Figure 12 (d, e, f) respectively.

Detection Configuration: The detection configuration used in our evaluation are shown in Table 15 (Section Appendix A). For all datasets, we have conducted a preliminary controlled experiment of detection parameters (see RQ4). This is to determine the best parameter for backdoor detection using AEGIS, without overfitting. For each dataset, the epsilon (ϵ) ball for input perturbation is fixed. For MNIST and Fashion-MNIST, the parameter ϵ is 100 and it is 500 for CIFAR-10. This places a uniform limit on input perturbation for each dataset. The perplexity for t-SNE is a tuneable parameter that balances the attention between the local and global aspects of the data. The authors suggest a value between five and 50 (Maaten and Hinton, 2008) and as a result we chose 30. The bandwidth in the mean shift algorithm is the size of the kernel function. This value is constant for each dataset, it is automatically computed with the scikit-learn mean shift clustering algorithm.⁴ For the backdoor attacks, the resulting bandwidths are 35, 28 and 21 for MNIST, Fashion-MNIST and CIFAR-10, respectively. Additionally, we also test the sensitivity of the AEGIS technique to vari-

⁴ https://scikit-learn.org/stable/modules/generated/sklearn.cluster.estimate_bandwidth.html

ance in the bandwidth, and (the number of) initial seed images (see **RQ4**). For instance, we run AEGIS with ± 3 around the respective calculated values for mean shift bandwidth.

Evaluation Metrics: We measure the performance of the backdoor injection attack by computing the *classification accuracy* on the testing data. We compute the *attack success rate* (ASR) by applying the trigger to all test images and measuring the number of modified images that are classified to the attack target label, i.e., classified to class seven (7). We also measure the *adversarial precision* of all robust models. In addition, we measure the classification accuracy of the clean adversarially robust models as a baseline for comparison. We also compare the performance of robust models (i.e., ASR and classification accuracy) to that of standard backdoored (and clean) models. For detection efficacy, we report the *number of feature representation clusters* found for all classes of all robust models.

5.2. Experimental Results

RQ1 - Attack Success Rate (ASR):

In this section, we present the effectiveness of backdoor injection attack. We illustrate that backdoors can be effectively injected in robust models without significantly reducing the classification accuracy and adversarial precision of the models. [Table 5](#) highlights the attack success rate (ASR), classification accuracy and adversarial precision of each trained model.

In our evaluation, we found that *robust models are highly vulnerable to backdoor attacks*. Backdoor attacks effectively caused the misclassification of 67.8% of backdoor-infected images to the attacker selected target labels, across all datasets and attack types (see [Table 5](#)). *Visible backdoor triggers are generally more effective than invisible backdoor triggers*, visible triggers are 2.5 times more successful than invisible triggers (see *attack success rate* (“ASR”) in [Table 5](#)). Specifically, visible triggers effectively caused the misclassification of 96.4% of backdoor-infected images to the attacker selected target labels, in comparison, invisible triggers caused the misclassification of only 39.3% of infected images to the target class (see [Table 5](#)). These results suggest that backdoor injection attacks are highly effective on robust models.

Robust DNNs are highly susceptible to backdoor attacks, with a 67.8% attack success rate (ASR), on average.

In our experiments on robust optimization via adversarial training (AT), we observed that *robust models are less susceptible to backdoor attacks than standard models*. Backdoor attacks are more successful on standard models than robust models because adversarial perturbations introduced during adversarial training may influence the shape and dimension of the backdoor trigger. We found that a backdoor attack is 12% more effective on a standard DNN model than on a robust model, with ASR of 67.83% and 75.86% for a robust and standard backdoor-infected model, on average, respectively (see [Table 5](#)). This result holds across attack types and regardless of the stealthiness (or visibility) of the backdoor trigger. For instance, the ASR for invisible static perturbations is 30.7% on robust models, in comparison to 60.4% on standard models. Our results imply that backdoor attacks are more effective in a standard model than a robust model (resulting from AT).

Backdoor attacks are (12%) more effective on standard DNN models than robust models obtained via adversarial training.

Backdoor injection in robust DNNs does not cause a significant reduction in adversarial precision. Backdoor injection in robust models only reduced adversarial precision by about 3.7%, in comparison to clean robust models. Backdoor-infected robust models have an adversarial precision of 83.21% on average, while clean robust models have an adversarial precision of 86.37% on average (see “Adv. Prec.”

in [Table 5](#)). In particular, the adversarial precision of robust models injected with visible triggers (86.14%) is comparable to that of clean robust models (86.37%). This result suggests that backdoor injection has little or no effect on the adversarial precision of infected robust models.

Backdoors do not significantly reduce the adversarial precision of robust models, they caused only 3.7% reduction, on average.

In our evaluation, backdoor injection in robust DNNs does not cause a significant reduction in classification accuracy for clean images. Overall, backdoor-infected robust models have about 2.6% reduction in classification accuracy in comparison to clean robust models, on average. Despite backdoor injection, robust models still achieved a high classification accuracy (91.55%) for clean images, on average (see “Class. Acc.” in [Table 5](#)). In comparison, clean robust models achieved a 93.96% classification accuracy. This is not a significant reduction in classification accuracy. In particular, models trained with visible triggers maintained a higher classification accuracy than models trained with invisible triggers. Models trained with visible triggers had a classification accuracy of 93.80% while models trained with invisible triggers had a lower classification accuracy of 89.30% (see [Table 5](#)). These results imply that backdoor injection in robust models does not significantly influence the classification accuracy of clean images.

Robust backdoor-infected models maintain a high classification accuracy (83.21%), on average.

RQ2 - Detection Effectiveness: In this section, we evaluate the efficacy of our backdoor detection approach (AEGIS). Specifically, we evaluate the technique’s efficacy in (a) detecting backdoor-infected robust models, and (b) revealing the backdoor-infected class, for both visible and invisible backdoor triggers. Furthermore, we demonstrate that AEGIS is *specialized to detecting backdoors in robust models* by showing it is ineffective on standard (non-robust) models. In particular, we show that AEGIS did not detect a backdoor for clean standard models, and backdoor-infected standard models.

Visible Backdoor Trigger: In our evaluation, AEGIS effectively detected all visible backdoor-infected robust DNNs, for both localised and distributed backdoors, and all classification tasks. It accurately detected all backdoor-infected models by identifying classes that have more than two feature clusters for the training set and the translated image set. The results showed that all clean untargeted classes of backdoor-infected robust models, as well as all classes of clean robust models have exactly two clusters, while, all targeted classes of backdoor-infected models have more than two clusters. These imply that AEGIS detected all robust models infected with visible backdoor triggers and the corresponding target class. Additionally, there are no false positives. This means that a clean model is not incorrectly predicted as a backdoor-infected model (see [Table 6](#)).

In particular, for each targeted class, the mean shift clustering of the features of the backdoor-infected models reveals these models consistently have more than two clusters (see [Figure 14](#) in the Appendix). Notably, these clusters include one cluster for the clean training images and at least two clusters for the translated images. The clusters for the translated images include at least one cluster capturing the image translation for the poisoned images, and another cluster for the translated clean images. Meanwhile, the clean untargeted classes have precisely two clusters of features, one for the training set and another for the translated image set. Likewise, for the clean robust models, each class has exactly two distinct clusters, one cluster for the training set and another cluster for the translated image set (see [Table 16](#) in the Appendix).

Table 6

Backdoor Detection Efficacy: $\checkmark$ indicates that AEGIS detected a backdoored-infected model/class and $\times$ indicates that AEGIS did not (or failed to) detect the presence of a backdoored model/class, e.g., in clean models (or stealthy static invisible backdoor-infected models).

	MNIST					Fashion-MNIST					CIFAR-10				
	Backdoor-Infected				Clean	Backdoor-Infected				Clean	Backdoor-Infected				Clean
	Visible		Invisible			Visible		Invisible			Visible		Invisible		
	Local	Dist	Static	Adv		Local	Dist	Static	Adv		Local	Dist	Static	Adv	
Backdoor Detection	✓	✓	✗	✓	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓	✗
Backdoor Class Detection	✓	✓	✗	✗	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓	✗
False Positive Class Detection	0	0	0	1	0	0	0	3	1	0	0	0	0	1	0

Table 7

Effectiveness of AEGIS on standard (non-robust) models. We show the number of clusters produced by AEGIS for each class, using a clean standard CIFAR-10 model and a CIFAR-10 model poisoned with a visible localized backdoor trigger. The number of clusters produced by AEGIS for the undetected poisoned classes (i.e., two clusters for the poisoned class (7)) is in **bold**. $\times$ indicates the backdoor-infected class/model is undetected, and "N/A" means "not applicable".

Standard (non-robust) Model Settings		Number of clusters produced by AEGIS										Backdoor detected	
Dataset	Backdoor Trigger	Benign classes					Poisoned class					Class	Model
		0	1	2	3	4	5	6	8	9	7		
CIFAR-10	Clean	2	2	2	2	2	2	2	2	2	2	N/A	N/A
	Localized (Visible)	2	2	2	2	2	2	2	2	2	2	$\times$	$\times$

AEGIS effectively detected all (100%) visible trigger backdoored robust DNNs.

Invisible backdoor triggers: Our evaluation results show AEGIS detected five (out of six) invisible backdoor-infected robust DNNs. Specifically, AEGIS was unable to detect the MNIST backdoor model with the invisible static trigger. It accurately detected the backdoor-infected models by identifying classes that have more than two feature clusters for the training set and the translated image set. In terms of the detection of the target backdoored class, AEGIS is able to detect the targeted backdoor class in four out of the six models with invisible backdoors. AEGIS is unable to detect the target class for the MNIST backdoor model with the adversarial static trigger (see Table 6). Additionally, for some of the backdoor models AEGIS detected more than two clusters for the non-targeted classes (see Table 17 in the Appendix). On average, AEGIS detected a non-targeted class as a backdoored class (false positive detection) 11.1% of the time (see Table 6).

AEGIS accurately identified the infected class, for all classification tasks and both visible trigger backdoor attacks (see Table 6). The mean shift feature clustering of each class in the backdoor-infected model reveals that only the infected class had more than two clusters, with one cluster for the training set and at least two clusters for the translated images. For invisible backdoor attacks, AEGIS identified five out of six backdoored models and four out of the six targeted classes.

Overall, AEGIS detected 91.6% (11/12) of backdoor-infected models, across all (12) tested configurations.

Standard (Non-robust) Models: In this experiment, we investigate if AEGIS detects backdoors in standard (non-robust) models using two standard CIFAR-10 models, namely one clean model, and a poisoned model injected with a localized backdoor trigger.

We found that AEGIS is specialized for backdoor detection in robust models: It is ineffective in detecting backdoors in standard models. Table 7 provides details of the effectiveness of AEGIS on clean and backdoor-infected standard models. AEGIS correctly predicts clean standard models as benign, i.e., a clean model is not incorrectly predicted as a backdoor-infected model. However, AEGIS does not detect a backdoor-infected model or class for the poisoned model. Specifically, AEGIS produces exactly two clusters for all classes in both models including the poisoned class in the

backdoor-infected model (see Table 7). Thus, AEGIS does not detect the backdoor-infected model or the poisoned class for standard (non-robust) models. These results imply that AEGIS is not directly amenable to standard models. Even though AEGIS has no false positives (it does not incorrectly classify a clean standard model), it is unable to detect a backdoor-infected standard model. This is expected since AEGIS expects a data distribution typically found in robust models. Unlike standard models, robust models have a different data distribution. In particular, AEGIS is designed to handle the resilience of robust models to perturbations introduced during adversarial training, and such perturbations are uncommon in standard models.

AEGIS is specialized for backdoor detection in robust models. It is not effective in detecting backdoors in standard (non-robust) models.

RQ3 Comparison to the state of the art. In this section we compare our backdoor detection approach (AEGIS) to the state of the art backdoor detection technique called Neural Cleanse (NC) (Wang et al., 2019). NC is a reverse engineering approach that assumes the reverse engineered trigger for the backdoor-infected class is smaller than the median size of the reverse engineered trigger for all classes. Specifically, NC's outlier detector identifies a class as backdoor-infected (with 95% probability) if it has an anomaly index that is larger than two. Although, this assumption holds for standard models because the underlying distribution of data points is normal (Wang et al., 2019), it does not hold for robust models. Due to the unbrutle nature of robust models (Madry et al., 2018), the underlying distribution of data points does not form a normal distribution because of adversarial perturbations introduced during robust training.

To compare NC and AEGIS, we run NC to detect localised backdoors in a standard model and a robust model. First, we train standard and robust models for CIFAR-10 that are poisoned with localised backdoors (using the backdoor injection process described in Section 4). We then reverse engineer the trigger for both the standard and robust backdoor-infected models using projected gradient descent on 100 random images from the training set (Madry et al., 2018), using the default NC detection parameters for both the standard and robust models. Next, we estimate the anomaly index for each class, i.e., the size of the trigger for each

Table 8

Details of the parameter sweep with varying epsilon (ϵ) values ($\epsilon \in \{1-7\}$) for Neural Cleanse. We show the anomaly indices produced by Neural Cleanse for each class, using a CIFAR-10 robust model poisoned with a visible localised backdoor trigger. Anomaly indices for undetected poisoned classes (i.e., anomaly index less than two for the poisoned class (7)) are in **bold**, as well as the results for the default parameter setting ($\epsilon = 4.0$). $\times$ indicates the backdoor-infected class/model is undetected by NC, and $\checkmark$ means the backdoor-infected class/model is detected by NC.

Detection Setting	Anomaly indices produced by Neural Cleanse										Backdoor detected	
	Benign classes									Poisoned class	Class	Model
Epsilon (ϵ)	0	1	2	3	4	5	6	8	9	7		
$\epsilon = 1.0$	0.625	0.739	0.514	1.506	0.127	0.400	1.228	0.724	0.127	1.825	$\times$	$\times$
$\epsilon = 2.0$	0.084	0.148	1.142	0.084	0.383	1.537	0.760	1.214	0.589	1.611	$\times$	$\times$
$\epsilon = 3.0$	0.475	0.373	0.796	0.291	0.291	1.133	0.733	0.852	0.620	0.729	$\times$	$\times$
$\epsilon = 4.0$	0.382	0.545	0.832	0.304	0.304	0.686	0.957	1.046	0.670	0.679	$\times$	$\times$
$\epsilon = 5.0$	0.670	0.679	1.224	0.187	0.187	0.448	1.248	1.453	0.484	1.293	$\times$	$\times$
$\epsilon = 6.0$	0.823	0.526	1.417	0.349	0.097	0.185	1.079	1.172	0.097	0.967	$\times$	$\times$
$\epsilon = 7.0$	0.793	0.459	1.903	0.556	0.495	0.266	1.080	1.187	0.266	0.934	$\times$	$\times$

Table 9

Details of the parameter sweep with varying step-size values (step-size $\in \{1-7\}$) for Neural Cleanse. We report the anomaly indices produced by Neural Cleanse for each class, using a CIFAR-10 robust model poisoned with a visible localised backdoor trigger. Anomaly indices for undetected poisoned class (i.e., anomaly index less than two for the poisoned class (7)) are in **bold**, as well as the results for the default parameter setting (step-size = 4.0). $\times$ indicates the backdoor-infected class/model is undetected by NC, and $\checkmark$ means the backdoor-infected class/model is detected by NC.

Detection Setting	Anomaly indices produced by Neural Cleanse										Backdoor detected	
	Benign classes									Poisoned class	Class	Model
step-size	0	1	2	3	4	5	6	8	9	7		
step-size= 1.0	0.627	0.781	1.140	0.197	0.082	0.082	0.972	0.921	0.114	0.722	$\times$	$\times$
step-size= 2.0	0.809	0.398	1.086	0.181	0.181	0.554	0.795	0.844	0.208	0.944	$\times$	$\times$
step-size= 3.0	0.319	0.682	1.033	0.281	0.281	0.620	1.232	1.334	0.667	0.817	$\times$	$\times$
step-size= 4.0	0.357	0.529	0.712	0.543	0.357	0.727	0.877	0.981	0.637	0.716	$\times$	$\times$
step-size= 5.0	0.402	0.688	0.763	0.477	0.402	0.661	1.147	1.286	0.825	0.617	$\times$	$\times$
step-size= 6.0	0.306	0.590	0.806	0.297	0.297	0.802	1.253	1.204	0.663	0.686	$\times$	$\times$
step-size= 7.0	0.288	0.490	0.793	0.316	0.288	0.872	1.075	1.087	0.665	0.684	$\times$	$\times$

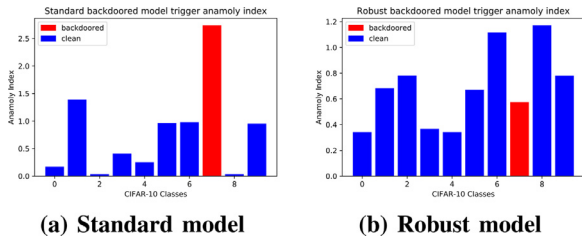


Fig. 13. Anomaly indices for the reverse engineered triggers for backdoor-infected standard and robust models.

class by measuring the average L_1 norm deviation from the original images to the reverse-engineered images (this is equivalent to counting the number of pixels changed). The mean L_1 norms are shown in Figure 18. Additionally, we repeat the same experiment for six varying values (range = ± 3) for each detection parameter (i.e., epsilon $\epsilon \in \{1, 2, 3, 5, 6, 7\}$ and step-size = $\{1, 2, 3, 5, 6, 7\}$) to ensure the obtained results are not due to NC's sensitivity to detection parameters. Table 8 and Table 9 highlight the results for NC's effectiveness for varying values of epsilon (ϵ) and step-size, respectively.

Our evaluation results show that NC detects the poisoned class for standard models, but it fails to accurately detect the poisoned class for robust models. This result holds for all tested detection parameter configurations. Particularly, Table 8 and Table 9 show that NC does not detect the poisoned class for (14) different parameter settings. In contrast, AEGIS detected the backdoor-infected robust model as well as the poisoned class (see RQ2). Figure 13 shows the anomaly indices for each class, i.e., the estimated size of the reverse engineered trigger, for a standard backdoor-infected model (a) and for a robust backdoor-infected model (b). The red bar

represents the anomaly index for the backdoor-infected class. We found that on standard models, the size of the backdoor-infected class is small and it is indeed detected as anomalous by NC, i.e., the anomaly index of the poisoned class (class seven (7)) is greater than two (2) (see Figure 13(a)). However, on robust models, NC fails to detect the poisoned class as anomalous. In fact, the anomaly index of the backdoor-infected class in the robust model is significantly less than two (see Figure 13(b)). This result suggests that while NC is suitable for backdoor detection in standard models, it is not suitable for detecting backdoor in robust models.

The state-of-the-art backdoor defense (Neural Cleanse) fails to accurately detect the backdoor-infected class for robust models, for (14) different detection settings with varying values of epsilon (ϵ) and step-size.

RQ4 - Sensitivity Analysis of Detection parameters: We evaluate the sensitivity of AEGIS to varying values of the detection parameters, i.e., epsilon (ϵ), mean shift bandwidth and (number of) initial seed images.⁵ We evaluate the sensitivity of these parameters for all attacks and data sets. For these parameters, we report the detection accuracy and the false positive rate for all tested values of these detection parameters. Although the mean shift bandwidth was automatically computed using the scikit-learn mean shift clustering algorithm, we still examined the sensitivity of the resulting values with a variance of ± 3 . For MNIST and FMNIST dataset, we experimented with varying epsilon values of ± 40 around the default value of 100 used, i.e., between 60 and 140, in particular, $\epsilon \in \{60, 70, 80, 90, 100, 110, 120, 130, 140\}$. For CIFAR-10, we experiment with varying epsilon values of

⁵ We do not evaluate the sensitivity of the t-SNE perplexity parameter, because this has been shown to be robust between values five and 50 (Maaten and Hinton, 2008).

Table 10
Sensitivity to Detection Parameters (“#” = “Number of”)

Detection Parameters	#Config	#Detection Accuracy (#)	#Failure Rate (#)	#False Positive Rate (#)
Epsilon (ϵ)	54	98.1% (53)	1.9% (1)	0% (0)
Meanshift bandwidth	18	94.4% (17)	5.6% (1)	1.2% (2)
#Images	42	88.1% (37)	11.9% (5)	2.11% (8)
Stability	30	90% (27)	10% (3)	0.7% (2)

± 200 around the default value of 500 used, i.e., between 300 and 700 ($\epsilon \in \{300, 350, 400, 450, 500, 550, 600, 650, 700\}$). For all datasets, we vary the number of initial sample images ± 300 around the default value of 500 used, i.e., between 200 and 800 ($\{200, 300, 400, 500, 600, 700, 800\}$). We also study the stability of AEGIS’ detection by executing five runs for each robust model that has been infected with the visible backdoor trigger.

The epsilon sensitivity results showed that AEGIS has a very low sensitivity to varying values of epsilon. For all values of epsilon, AEGIS could identify a backdoor-infected model and the poisoned class for 98% (53 out of 54 configurations) of all configurations, with no false positives (see Table 10). One backdoor-infected model was undetected, specifically, the distributed backdoor attack on MNIST at $\epsilon = 60$. We found that for the MNIST distributed backdoor attack, the epsilon value at 60 is too low. Thus, we recommend that higher epsilon (ϵ) values be used for (distributed) backdoor detection.

For all values of epsilon (ϵ), AEGIS detected 98% of the backdoor-infected models, with no false positives.

For mean shift sensitivity, our evaluation revealed that AEGIS has a very low sensitivity to varying values of the mean shift bandwidth. AEGIS detected 94% of the backdoored model for all mean shift configurations, i.e., 17 out of 18 configurations (see Table 10). In particular, for all tested mean shift values, AEGIS did not detect a backdoored model for one value of the mean shift bandwidth. Specifically, such a mean shift value is 24 for the CIFAR-10 model poisoned with distributed backdoor. This result suggests that for values higher than the computed mean shift bandwidth value, AEGIS may not detect the backdoor-infected class. Besides, AEGIS reported two false positives. In both cases a benign class other than the poisoned class was also misclassified as backdoored by AEGIS. Specifically, false positives were manifested for MNIST localised backdoored and CIFAR-10 distributed backdoored models, both with mean shift bandwidth values less than the computed values. Hence, we recommend to use the computed mean shift bandwidth value for accurate backdoor detection.

AEGIS has a 94% detection accuracy and a 1.2% false positive rate, for all tested mean shift bandwidth values.

For the sensitivity of AEGIS to the number of initial seed images, our investigation reveals that AEGIS has a fairly low sensitivity to varying values of the number of initial images. AEGIS detected 37 (88.1%) out of 42 tested configurations of varying number of initial seed images. Specifically, the five configurations AEGIS is unable to detect backdoors includes the MNIST localised model where the number of initial images is 300, as well as poisoned CIFAR-10 models where the number of initial images are 200 and 400 for the localised backdoors, and 200 and 300 initial images for the distributed backdoors. Overall, AEGIS has a low false positive rate of only 2.1% (see Table 10). Hence we recommend, using at least 500 initial seed images for effective detection of backdoors.

AEGIS has 88.1% detection accuracy and 2.1% false positive rate, for varying number of initial seed images.

Our experiments reveal that AEGIS is a fairly stable algorithm. To evaluate the stability of AEGIS we run the full technique five

times independently on MNIST, Fashion-MNIST and CIFAR-10 models with visible backdoor triggers. We find that out of the 30 runs, AEGIS can detect the backdoor 27 times (90%). AEGIS did not detect two MNIST distributed backdoor runs and one CIFAR-10 distributed backdoor. The false positive rate is extremely low at 0.74%. For maximum effectiveness, we recommend multiple runs of the AEGIS technique.

AEGIS is a fairly stable algorithm with a 90% detection rate and low false positive rate of 0.74%.

RQ5 - Attack Comparison: In this section, we compare the effectiveness of all four backdoor attack triggers namely the visible triggers (i.e., localised and the distributed triggers) as well as the invisible triggers (static perturbation and adversarial triggers). Specifically, we compare their attack success rate, and their effect on the classification accuracy and adversarial accuracy of the robust model. We also examine the detection efficacy of AEGIS on each backdoor trigger. Table 5 highlights the attack success rate (ASR), classification accuracy and adversarial precision of each backdoor trigger.

First, let us compare the effectiveness of backdoor attack triggers based on their stealthiness (i.e., visibility). Our results show that robust DNN models are less susceptible to invisible triggers (see “ASR” Table 5). In addition, we found that visible triggers have less impact on the adversarial precision or classification accuracy of robust models, in comparison to invisible triggers. Robust models injected with visible backdoor triggers have similar adversarial precision and classification accuracy to clean robust models (see “Adv. Prec.” and “Class. Acc.” in Table 5). Meanwhile, in comparison to clean robust models, invisible triggers reduce the classification accuracy and adversarial precision of robust models by 5% and 7%, respectively. These results suggest that the stealthiness (i.e., visibility) of a backdoor trigger influences the effectiveness of the attack, in particular, visible triggers are more effective than invisible triggers.

Visible triggers are more effective and have less impact on the (adversarial) accuracy of robust models than invisible triggers.

We compare the effectiveness of the two visible backdoor attack triggers based on the specific trigger types, i.e., localised vs distributed. We found that the distributed backdoor attack is more effective than the localised backdoor attack, it has a higher attack success rate. The distributed attack is 6.95% more successful than the localised backdoor attack, on average (see Table 5). Additionally, the distributed backdoors have a higher classification accuracy than the localised backdoors, albeit only a slight improvement of 0.12%. Overall, the distributed backdoors performed better than the localised backdoors.

The distributed backdoor attack is (6.95%) more effective than the localised backdoor attack on robust models, on average.

Let us compare the effectiveness of the two invisible backdoor triggers, i.e., the static and adversarial perturbation. Table 5 shows that adversarial perturbation is 56% more effective than the static invisible perturbation, with 48% vs 31% ASR, on average (see Table 5). This is because the adversarial perturbation (trigger) is dynamic and more powerful, it is derived from both the model and

Table 11
AEGIS Efficiency in terms of detection runtime.

Dataset	AEGIS Runtime			
	Visible Backdoor		Invisible Backdoor	
	Localised mins (secs)	Distributed mins (secs)	Static mins (secs)	Adversarial mins (secs)
MNIST	5.08 (304.5)	5.18 (310.5)	5.36 (321.5)	5.24 (314.3)
Fashion-MNIST	5.36 (321.5)	5.32 (319.4)	5.28 (317.3)	5.11 (306.8)
CIFAR-10	9.39 (563.5)	9.34 (560.6)	9.29 (557.9)	9.36 (561.7)

sample images from the dataset. Besides, the adversarial precision and classification accuracy of both triggers are similar. This result suggests that the quality of the invisible trigger influences the effectiveness of invisible backdoor attacks.

Invisible adversarial backdoor triggers are significantly more effective (56%) on robust models than static backdoor triggers.

In our evaluation, AEGIS detects 91.6% of backdoor attacks (i.e., 11 out of 12 tested backdoor-infected models). For both visible attacks, AEGIS detected the infected class in addition to the backdoor-infected model (see Table 6). For invisible backdoors, AEGIS was able to detect five out of the six backdoored models and four out of six poisoned classes. (see Table 6). We find that invisible backdoor attacks are *slightly more stealthy* in comparison to visible backdoor attacks.

AEGIS detects 91.6% (11 out of 12) of backdoor-infected models, across all attack types (visible and invisible).

RQ6 AEGIS Efficiency. We evaluate the detection time of AEGIS, i.e., the time taken to run the AEGIS technique on a backdoor-infected model. Table 11 shows the time taken for each attack type and dataset.

AEGIS is very efficient; it took five to nine minutes to run on average on a backdoor-infected model. In contrast, the state of the art defenses (for standard models) are known to take hours to days to detect a backdoor-infected model (Gao et al., 2019; Wang et al., 2019). Furthermore, we observed that the time taken by AEGIS increases as the complexity of the model and dataset increases (see Table 11). For instance, AEGIS took almost twice the time taken to run on MNIST models (five minutes) to run on CIFAR-10 (nine minutes). In addition, there is no significant difference in the time taken to detect each attack type, i.e., localised/distributed backdoor (visible trigger) or static/adversarial trigger (invisible trigger) (see Table 11). These results illustrate that AEGIS is computationally efficient and its efficiency is not adversely affected by the backdoor attack type.

AEGIS was reasonably fast, it took five to nine minutes to run on a backdoor-infected model.

RQ7 Ablation Study. Let us evaluate the effect of our design choices on the effectiveness of AEGIS, especially in comparison to alternative design choices. The goal is to investigate how our design choices compare to closely-related, alternative methods. Particularly, we examine AEGIS's use of t-SNE for dimensionality reduction and data visualization, as well as its use of mean shift clustering. In this RQ, we employed two robust models trained for CIFAR-10 dataset that are poisoned with localized and distributed visible backdoors. Table 12 and Table 13 highlight the comparison of the design choices of AEGIS to alternative design choices in terms of dimensionality reduction and clustering, respectively.

Dimensionality Reduction and Data Visualization: In this experiment, we examine the effectiveness of AEGIS with t-SNE, our default dimensionality reduction algorithm (called AEGIS-t-SNE), in comparison to replacing t-SNE with Principal Component Analy-

sis (PCA) (called AEGIS-PCA). We compare to PCA as an alternative since it is the most popular dimensionality reduction technique for sparse datasets (Maćkiewicz and Ratajczak, 1993). Besides, other dimensionality reduction alternatives are not amenable to our goal since they have strong assumptions about the underlying data distribution, e.g., Uniform Manifold Approximation and Projection for Dimension Reduction (UMAP) (McInnes et al., 2018) assumes uniform data distribution.

Our experimental results show that the t-SNE algorithm is more effective in backdoor detection than the PCA algorithm for AEGIS. We found that the default setting of AEGIS (i.e., AEGIS-t-SNE) is more effective than using PCA (AEGIS-PCA). Table 12 highlights the clustering provided by default AEGIS (i.e., AEGIS-t-SNE) versus AEGIS-PCA, for both the localised and distributed backdoor-infected robust models. While AEGIS-t-SNE detected the backdoored model and class, we found that AEGIS-PCA does not detect the backdoored model or the backdoored class: For both localised and distributed backdoored robust models, AEGIS-PCA does not detect the backdoored model or class, indeed, it produces two clusters for all classes, including the backdoored class (seven). We attribute the poor performance of AEGIS-PCA to the non-linearity of the dataset, since PCA is more effective when dealing with linear data. This result demonstrates that t-SNE is vital to the effectiveness of AEGIS and it is appropriate for backdoor detection in robust models.

For AEGIS, t-SNE (AEGIS -t-SNE) is more effective in backdoor detection than PCA (AEGIS -PCA): Unlike default AEGIS (AEGIS -t-SNE), AEGIS -PCA does not detect the backdoor-infected model or the poisoned class.

Clustering: To evaluate our choice of clustering algorithm, we examine the effectiveness of default AEGIS with mean-shift clustering (called AEGIS-MS) to two closely-related alternatives to mean-shift clustering, namely affinity propagation (Dueck, 2009) and HDBSCAN (McInnes et al., 2017). Specifically, we compare the default AEGIS (AEGIS-MS), with replacing mean-shift clustering with affinity propagation (called AEGIS-AP) or HDBSCAN (called AEGIS-HDBSCAN). We chose these two clustering algorithms because they are state-of-the-art clustering methods that are closely related to mean shift clustering. Besides, they do not require prior knowledge of the (expected or desired) number of clusters unlike alternatives such as K-means (Likas et al., 2003), spectral clustering (Ng et al., 2001) or agglomerative clustering (Gowda and Krishna, 1978).

On one hand, mean shift clustering is more effective for detecting backdoors than affinity propagation for AEGIS. Specifically, AEGIS using affinity propagation (AEGIS-AP) does not detect a backdoor-infected robust model or a poisoned class, but default AEGIS with mean shift clustering (AEGIS-MS) detects both the backdoor-infected model and the poisoned class. Table 13 shows that unlike default AEGIS (AEGIS-MS), AEGIS-AP does not detect a backdoor-infected robust model or a poisoned class. This result suggests that affinity propagation clustering is not suitable for AEGIS, i.e., AEGIS-AP is not amenable to backdoor detection. We attribute the poor performance of AEGIS-AP to the fact that affinity propagation is inherently a partitioning algorithm which causes its resulting clusters to be easily polluted by noisy or distant data points, such noisy data points are lumped into nearby clusters using affinity propagation. This is particularly a problem for backdoor detection especially for the poisoned class since clusters belonging to the poisoned data points are evidently lumped with the clusters of benign data points (see Table 13).

Affinity propagation is not a viable alternative to mean shift clustering for AEGIS. Unlike default AEGIS (AEGIS -MS), AEGIS using affinity propagation (AEGIS -AP) does not detect a backdoor-infected model or the poisoned class.

Table 12

AEGIS-t-SNE versus AEGIS-PCA: Evaluation of Dimensionality Reduction/Visualization Design Choice using localised and distributed CIFAR-10 visible backdoor-infected robust models. True Positives (i.e., correct detection of the poisoned class) are in **bold**.

Dataset	Type of	Detection Setting	Number of Resulting Clusters										Backdoor detected	
			Benign classes									Poisoned class		
			0	1	2	3	4	5	6	8	9	7	Class	Model
CIFAR-10	Localised	AEGIS-t-SNE	2	2	2	2	2	2	2	2	2	3	✓	✓
		AEGIS-PCA	2	2	2	2	2	2	2	2	2	2	✗	✗
	Distributed	AEGIS-t-SNE	2	2	2	2	2	2	2	2	2	3	✓	✓
		AEGIS-PCA	2	2	2	2	2	2	2	2	2	2	✗	✗

Table 13

AEGIS-t-SNE vs. AEGIS-AP vs. AEGIS-HDBSCAN: Evaluation of Clustering Design Choice using localised and distributed CIFAR-10 visible backdoor-infected robust models. True Positives (i.e., correct detection of the poisoned class) are in **bold** and False Positives (i.e., incorrect detection of a benign class as poisoned) are in **bold italics** (e.g., AEGIS-AP).

Dataset	Type of	Detection Setting	Number of Resulting Clusters										Backdoor detected	
			Benign classes									Poisoned class		
			0	1	2	3	4	5	6	8	9	7	Class	Model
CIFAR-10	Localised	AEGIS-MS	2	2	2	2	2	2	2	2	2	3	✓	✓
		AEGIS-AP	19	19	34	17	196	21	73	23	145	105	✗	✗
		AEGIS-HDBSCAN	2	2	2	3	2	2	2	2	2	3	✓	✓
	Distributed	AEGIS-MS	2	2	2	2	2	2	2	2	2	3	✓	✓
		AEGIS-AP	20	116	104	104	72	54	53	20	143	21	✗	✗
		AEGIS-HDBSCAN	2	2	2	2	2	2	2	2	2	3	✓	✓

On the other hand, we found that *HDBSCAN* is almost as effective as mean-shift clustering for detecting backdoors in robust models. Table 13 shows that default AEGIS (AEGIS-MS) is comparable to replacing mean-shift clustering with HDBSCAN (AEGIS-HDBSCAN). Both clustering algorithms enable AEGIS to identify a backdoor-infected model and the poisoned class. Results show that AEGIS-HDBSCAN is effective in detecting backdoors in robust models: For both localized and distributed backdoor-infected CIFAR-10 models, AEGIS-HDBSCAN correctly identifies the poisoned class, except for the mis-identification of class three (3) as a poisoned class for the localised backdoor (see Table 13). Overall, these results demonstrate that HDBSCAN is similarly as effective as mean-shift clustering. The effectiveness of HDBSCAN is because similar to mean-shift clustering, it makes minimal assumptions about the underlying dataset. HDBSCAN does not partition the data, and it effectively leaves sparse or noisy data points as independent clusters, hence noisy data points (e.g., poisoned data points) are not lumped with the nearest cluster. Overall, this result suggests that HDBSCAN is an effective alternative to mean-shift clustering for AEGIS.

HDBSCAN is almost as effective as mean-shift clustering for AEGIS. Similar to default AEGIS (AEGIS-MS), AEGIS with HDBSCAN (AEGIS-HDBSCAN) detects the backdoor-infected model as well as the poisoned class.

5.3. Discussions and Future Outlook

In this section, we discuss concerns about the application of AEGIS, in particular, how an adaptive attack can evade the detection of AEGIS, and how implicit assumptions of AEGIS (w.r.t. data distribution) can be applied to fool it or influence its performance.

Counter-measures against AEGIS : An attacker that is aware of AEGIS's detection methodology can ensure that clean or backdoored models are trained in a manner that tricks AEGIS and reduces its effectiveness. For instance, instead of the typical backdoor data poisoning attack vector, a powerful attacker can train a backdoor-infected model such that the backdoor image mimics neuron output values (seen in clean models). This powerful at-

tack may evade the detection of AEGIS, such that instead of simply causing a mis-classification of the backdoored image, it fools AEGIS to believe the backdoor neuron representation is similar to the neuron representation of clean images. Likewise, an adaptive attacker can deceptively train clean models to reduce the accuracy of AEGIS. As an example, an attacker may fool AEGIS by ensuring that clean models (similar to backdoored models) also have more than one data distribution. Although, this attack does not affect the detection of backdoor models by AEGIS, it may cause false positives, where AEGIS also detects such deceptive clean models as backdoored models. In the future, we plan to investigate these more powerful attack vectors and explore potential defenses to protect against them beyond AEGIS and our current threat model.

Data Distribution assumption: In this work, we have assumed that *clean models have only one data distribution for each class label*, hence, AEGIS detects backdoors by examining if the backdoored model has more than one data distribution for a class label. Concretely, there is an implicit assumption in our method that the data corresponding to each label in the dataset contains data of only one distribution. Although, this assumption is valid within our threat model, it may not hold in other scenarios. As an example, consider a binary computer vision classifier which detects *dog* images, such that it has two classes or output labels, i.e. *dog*, and *not dog*. Consider that this classifier is trained on a dataset containing multiple animal images (e.g., cat, horse, rat etc.), which may correspond to multiple distributions for the *not dog* class. As a result, our data distribution assumption may not hold in this scenario. Besides, this assumption may lead to wrong detection of clean models as backdoored model, if the clean models also have multiple distributions, e.g., because of the limitations of the training (e.g., local optima or incomplete training), or the complexity of the task/dataset (e.g., for multi-label or multi-output classification). For instance, consider a classifier trained on a (fashion) dataset, where a data point (e.g., an image of a person wearing a piece of clothing) can be classified into multiple labels (e.g., the gender of the person, the type/size of clothing and the color of the cloth). In this scenario, the data distribution assumption may not hold for each label. As an example, consider the distribution of the “shirts”

clothing label, which may contain multiple data distributions representing different gender, sizes and colors of shirts. However, in our threat model the user has access to the training data, and can examine the data distribution before-hand (e.g., through methods such as t-SNE). Thus, the user can successfully analyze whether the number of distributions learned in the trained model correspond to the actual data, and if not there may be a backdoor distribution. In a different threat model where the user does not have access to the training data, they may have no means to verify such an assumption. However, note that AEGIS can still detect a backdoored model in the absence of this assumption, i.e., even when this data distribution assumption does not hold, we expect that the backdoored model still has multiple data distributions.

In a scenario where the user has no access to the training data for inspection, then the user may not be able to determine whether our data distribution assumptions holds or not. In this threat model, an attacker can further leverage this lack of data access to fool AEGIS, e.g., by ensuring that our data distribution assumption does not hold for clean models. As an example, an attacker can ensure that clean models are not properly trained, thus they converge to a local optima, hence, an *adaptive attacker* can ensure that trained clean models have more than one data distribution. Alternatively, the attacker can ensure that backdoored models also converge to a local optima with one data distribution. These attack vectors and threat models can fool AEGIS and reduce its effectiveness. However, we expect that such an adaptive attack will reduce the performance of the model (e.g., in terms of accuracy), since it forces the trained model to be sub-optimally trained. Furthermore, this assumption does not influence the ability of AEGIS to detect backdoor models, although it may cause a false positive detection (i.e., wrongly detect a clean model as a backdoored model because the clean model has more than one data distribution). In the future, we plan to investigate these line of attacks concerning alternative threat models and multiple data distributions, in order to develop potential defenses to mitigate these attack vectors.

Robustness Check: Since AEGIS is designed to detect backdoors in robust models, it assumes the analyzed model is robust, i.e., trained under robust optimization conditions. To ascertain an examined model is robust, there are automatic tests for adversarial robustness. For instance, [Madry et al. \(2018\)](#) demonstrates a white-box approach that inspects the last layer of a model to check if a model is robust. Likewise, a brute-force black-box test is to check the performance of the model on adversarial examples within the expected perturbation bound. Both of these approaches are reliable and easy to automate. Thus, using the aforementioned methods, it is possible to check for model robustness to determine the applicability of AEGIS for the model-at-hand. Besides, we have demonstrated that AEGIS is not applicable to standard models: It does not produce false positives for standard models and it does not detect backdoors in backdoor-infected standard models (see RQ2). Thus, in the absence of a robustness check, AEGIS does not detect a backdoor in a standard model (as intended) and it does not classify a clean standard model as backdoor-infected.

Alternative Threat Models: In the following we discuss changes in the threat model that may influence the performance of AEGIS in detecting backdoor in robust models. Firstly, this work assumes the *attacker trains on the entire training dataset* (see [Section 3](#)). However, an *adaptive attacker* can train on a subset of the training data such that the model still attains an acceptable performance. In this threat model, such an attack may make it more difficult for AEGIS to detect the backdoor-infected model or class. We expect that achieving an acceptable performance requires a substantial subset of the dataset that preserves the mixed data distribution hypothesis leveraged by AEGIS. However, in the future, we plan to investigate the potential of such adaptive attacks, how much they

may influence (e.g., decrease) the performance of AEGIS, and how to (extend AEGIS to) appropriately mitigate them.

We also assume an attacker directly poisons the training examples during outsourced model training, but a *sophisticated attacker* may have access to the data preparation pipeline such that he/she can directly poison the objects or scenes captured in the training images. This attack is much stronger than our default setting because of the potential *naturalness* of the resulting images. Given that the attacker only poisons for a targeted class and the mixed data distribution is preserved, we expect AEGIS to detect the backdoor in the resulting training images from such an attack. However, such a powerful attack may fool AEGIS because of the naturalness of the attack, especially if the attacker has sufficient resources to poison a substantial number of real-world objects/scenes (beyond the targeted class). Hence, such a “natural” attack may require new detection methods. We encourage researchers to investigate this “natural” backdoor attack vector and how to mitigate such attacks.

Finally, AEGIS is designed to be a white-box, post-training backdoor detection method. Hence, it may not be directly amenable to other threat settings, e.g., black-box, pre-training and in-training scenarios. For instance, consider scenarios that require detecting backdoors in poisoned datasets before (e.g., fingerprinting the information content in poisoned examples) or during the training process, AEGIS is not applicable in these scenarios since it requires white-box access to an already trained model and a clean training dataset. In addition, AEGIS may not be directly applicable to scenarios involving an ensemble of models or multiple datasets (e.g. federated learning). These scenarios are beyond the scope of this work and may require fundamentally different backdoor detection methods than AEGIS, since these threat models do not fulfill the requirements/assumptions of AEGIS. In the future, we plan to investigate these alternative threat models and how to mitigate backdoor in these settings. As an example, we plan to investigate the information content of backdoor examples to inform the automatic identification of backdoor in black-box, pre-training or in-training scenarios with no access to the model. In addition, we plan to investigate the impact of ensemble models and multiple data sources (e.g., federated learning settings) on the effectiveness of backdoor injection and detection. We also encourage researchers to develop practical approaches to defend and mitigate against backdoor in such alternative threat models.

In summary, similar to [Meng et al. \(2020\)](#) – a generic defense against adversarial attacks, we encourage researchers to investigate general defense mechanisms against backdoor that are applicable to several threat models. In the same vein, we plan to investigate generic defenses that are applicable to different threat models, e.g., with varying access levels including zero-knowledge, black-box, gray-box, and white-box.

6. Threats to Validity

Our evaluation is limited by the following threats to validity:

External validity: This refers to the generalisability of our approach and results. In particular, our findings may not generalize to other settings, different from the employed setting, specifically different (image classification) tasks, neural architectures, datasets and robust optimization methods. In the following, we discuss these threats/limitations in detail.

Tasks, Datasets and Class labels: There is a threat that our findings and approach (AEGIS) may not generalize to other classification tasks, datasets or more complex, larger labels. We have mitigated this threat by evaluating the performance of our approach using three major image classification tasks with varying levels of complexity (CIFAR-10, MNIST and FashionMNIST). These tasks have thousands of training and test images, providing confidence that our approach will work on similarly complex tasks and models.

Despite this mitigation, our findings are limited to these settings. Indeed, our findings may not be applicable to other object recognition datasets, other image classification tasks (e.g. image segmentation) and other classification tasks (e.g., image captioning). Besides, our experiments involve few (≤ 10) class labels, thus it may not generalize to models with a much larger number of labels or more complex labels (e.g., multi-label classification). In the future, we plan to investigate the applicability of AEGIS to different or more complex tasks. We also plan to develop generic approaches that are applicable across several tasks.

Neural Architecture and Robust Optimization: Our experiments were conducted using a specific neural architecture and robust optimization method, in particular, the ResNet architecture (He et al., 2016), and adversarial training (AT) (Madry et al., 2018). Hence, there is a threat that our findings do not generalize to simpler or more complex neural architectures where the model has less or more capacity. Besides, our findings may not generalize to other robust optimization methods beyond adversarial training. In the future, we plan to examine backdoor injection and defense across different neural architectures and robust optimization methods (e.g., adversarial defense via diversity and ensemble models (Abbasi and Gagné, 2017; Kariyappa and Qureshi, 2019; Li et al., 2021; Pang et al., 2019)). For a general evaluation of backdoor in robust models, we encourage researchers to employ a standard and wide range of adversarial defenses under different threat models (e.g., AutoAttack⁶ (Croce and Hein, 2020; 2021) and RobustBench⁷ (Croce et al., 2020)).

Internal validity: This concerns the correctness of our implementation of backdoor attacks and AEGIS' defense. This includes whether we have performed adversarial training rightly, accurately defined (in)visible backdoor triggers, successfully injected backdoors, and correctly implemented AEGIS. We mitigate this threat by thoroughly testing our implementations on sample images to ensure our implementation works as expected. In addition, we provide our implementation, datasets and results for replication and scrutiny.

Construct validity: It is possible that advanced backdoor triggers can be crafted to align to the input distribution of the training dataset. We mitigate this threat by ensuring that our backdoor triggers are similar to the ones described in the literature, as reported in previous related research. We emphasize that for robust models, the success and mitigation of backdoor attack variants such as blind backdoors (Bagdasaryan and Shmatikov, 2020), trojanning (Guo et al., 2020; Liu et al., 2018b; Zou et al., 2018) and adaptive attacks (Gao et al., 2019) are open research problems. These attacks have not been investigated for robust models. We consider the investigation of these advanced attacks against robust models as future work.

The backdoor detection scenario employed in this work is a threat to the construct validity of AEGIS. In our attacker model, the attacker injects backdoors in robust models via a *third-party platform* and the user has access to the clean training data, clean testing data and the trained robust model. We assume an attacker introduces poisoned examples into the training data when the model training is outsourced to a third-party. However, Li et al. (2020) has shown that there are alternative processes for injecting backdoors. Specifically, backdoors can also be injected via two alternative scenarios, i.e., (i) *third-party datasets* – where an attacker provides the poisoned dataset to users directly, or (ii) *third-party models* – where the attacker provides trained infected DNNs to the user (Li et al., 2020). We expect that AEGIS is applicable in the *third-party dataset* scenario since the user has the capabilities

required by AEGIS, i.e., access to the clean training data, clean testing data and white-box access. However, our approach (AEGIS) may not be directly applicable in the *third-party model* scenario since the user/defender may lack access to the training set and white-box access to the trained models. In the future, we shall investigate the injection and detection of backdoors in alternative attacker/defender scenarios.

Lastly, our design choices pose a threat to construct validity. Specifically, our use of *mean-shift clustering* and *t-SNE for dimensionality reduction* may influence the effectiveness of AEGIS. To mitigate this threat, we have conducted an ablation study investigating the effectiveness of our choices in comparison to alternative design choices (see RQ7).

7. Related Work

Robust Optimization: Adversarial attacks for Neural Networks (NNs) were first introduced in Szegedy et al. (2014). Researchers have introduced better adversarial attacks and built systems that are resilient to these attacks (He et al., 2017; Papernot et al., 2017; 2016a; 2016b). A significant leap has been made by introducing robust optimisation to mitigate adversarial attacks (Madry et al., 2018; Raghu et al., 2018b; Sinha et al., 2018; Wong and Kolter, 2018a). These defenses aim to guarantee the performance of machine learning models against adversarial examples. In this paper, we study the susceptibility of the models trained using robust optimisation to backdoor attacks. Then, we leverage the inherent properties of robust models to detect backdoor attacks.

In this work, we have studied backdoor detection/injection in robust models using adversarial training (AT) as the *only* robust optimization method. Even though adversarial training is an effective and well-known defense against adversarial examples (AE), there are other robust optimization techniques beyond adversarial training which may not be susceptible to backdoor detection or amenable to backdoor detection by AEGIS. A number of researchers have demonstrated that adversarial robustness can be achieved via an ensemble of diverse models (Kariyappa and Qureshi, 2019; Li et al., 2021; Pang et al., 2019), or by detecting unrecognised, potentially adversarial examples (Abbasi and Gagné, 2017). Indeed, we do not know the susceptibility of these robust optimization methods (except AT) to backdoor attacks, and AEGIS may not be sufficient to defend against backdoor injection in these settings. In the future, we plan to investigate the susceptibility of different robust optimization methods to backdoor attacks and how to effectively defend against them.

In addition, our findings and observations about backdoor attacks and AEGIS is *strictly empirical*. We provide no theoretical bound to the susceptibility of robust optimization to backdoor attacks or guarantees of AEGIS's defense. Indeed, it is vital to understand how our empirical observations relate to the robust optimization theory, e.g., in terms of the susceptibility of standard and robust models to backdoor attacks (see RQ1). In addition, it is interesting to know the lower and upper bound accuracy of AEGIS on certain poisoning attacks (e.g., (in)visible distributed or localized backdoor triggers), and how this relates to formulating backdoor defense as studying spurious/non-robust features in robust models (Ilyas et al., 2019). In particular, we encourage the theoretical investigation of how backdoor injection and defenses relate to robust optimization theory to provide mathematical insights into our empirical observations.

Backdoor Attacks: Backdoor attacks were introduced in BadNets (Gu et al., 2017), where an attacker poisons the training data by augmenting it. A pre-defined random shape (called trigger) is chosen for the attack. TrojanNN (Liu et al., 2018b) improves the attack by engineering the trigger and reducing the number of examples needed to insert the backdoor.

⁶ <https://github.com/fra31/auto-attack>

⁷ <https://robustbench.github.io/>

Yao et al. (2019) propose a transfer learning based backdoor. All of these attacks are visible to the human eye. Besides, other variants of backdoor attacks have also recently been developed such as blind backdoors Bagdasaryan and Shmatikov (2020), trojaning Guo et al. (2020); Liu et al. (2018b); Zou et al. (2018) and adaptive attacks Gao et al. (2019). In addition, Zhong et al. proposed a backdoor attacks where the trigger is hidden Zhong et al. (2020).

Li et al. (2020) provides a systematic literature review of backdoor attack mechanisms. This work demonstrates varying attack/threat models for backdoor attacks, for instance in terms of the access level of the attacker (e.g., access to training set, training schedule, model and/or inference pipeline). The paper also provides a taxonomy of poisoning based attacks (e.g., trigger properties such as target level, visibility, selection, appearance, and type (digital versus physical).) In this work (AEGIS), we have focused on the injection and detection of both invisible and visible backdoor attacks in robust models. The aforementioned attacks were demonstrated for standard models, not for robust training. To the best of our knowledge, we are the first to demonstrate the susceptibility of models trained under robust optimisation conditions (Madry et al., 2018) to (both visible and invisible) backdoor attacks.

Backdoor Detection and Mitigation: Several approaches have been developed to detect and mitigate backdoor attacks on standard machine learning models. Li et al. (2020) provides a comprehensive analysis of different defense mechanisms under different threat models.⁸ Table 1 compares an excerpt of the main characteristics of these approaches. These approaches can be categorized into three main types, namely, backdoor detection via (1) outlier suppression, (2) input perturbation and (3) model anomalies (Bagdasaryan and Shmatikov, 2020).

Outlier suppression based defenses prevent backdoored inputs from being introduced into the model (Du et al., 2019a; Hong et al., 2020). The main idea of these approaches is to employ differential privacy mechanism to ensure that backdoored inputs are under-represented in the training set. Unlike these approaches, our approach is not a training-time defense, rather the focus of our approach is to detect models that are already poisoned with backdoored inputs.

Input perturbation methods detect backdoors by attempting to reverse engineer small input perturbations that trigger backdoor behavior in the model. Such approaches include Neural Cleanse (NC) (Wang et al., 2019), ABS (Liu et al., 2019), TABOR (Guo et al., 2019), STRIP (Gao et al., 2019), NEO (Udeshi et al., 2019), DeepCleanse (Doan et al., 2019), AD (Xiang et al., 2020) and MESA (Qiao et al., 2019). In this paper, we focus on comparison to Neural Cleanse (NC) (Wang et al., 2019), we used NC as the representative backdoor defense. We compare our approach to NC (see RQ3), since NC is the state of the art and it has realistic defense assumptions (similar to AEGIS) (see Table 1). In particular, NC relies on finding a fixed perturbation that mis-classifies a large set of inputs, but since robust models are designed to be resilient to exactly such perturbations, we show that NC is inapplicable for robust models.

Model anomaly defenses detect backdoors by identifying anomalies in the model behavior. Most of these techniques focus on identifying how the model behaves differently on benign and backdoored inputs, using model information such as logit layers, intermediate neuron values and spectral representations. These approaches include SentiNet (Chou et al., 2018), spectral signatures (Tran et al., 2018), fine-pruning (Liu et al., 2018a), NeuronInspect (Huang et al., 2019), activation clustering (Chen et al., 2019), SCAN (Tang et al., 2019), NNoculation (Veldanda et al., 2020) and MNTD (Xu et al., 2019). However, unlike our approach, none of

these techniques detect backdoors in robust models. Additionally, SCAN (Tang et al., 2019), SentiNet (Chou et al., 2018), activation clustering (Chen et al., 2019) and spectral signatures (Tran et al., 2018) assume access to the poisoned dataset – an impractical assumption for backdoor defense (see Table 1). Moreover, fine-pruning (Liu et al., 2018a) is shown to be ineffective in existing work (Wang et al., 2019) and NNoculation (Veldanda et al., 2020) and MNTD (Xu et al., 2019) require training a shadow model for defense, leading to a computationally inefficient process. In contrast, AEGIS is computationally efficient, it does not require access to the poisoned dataset and it accurately detects backdoor-infected robust models.

Unlike the aforementioned works, we rely on the *clustering of feature representations in robust models* to detect backdoor attacks. Like our approach, Chen et al. (2019) employs feature clustering to detect backdoors in standard DNNs; it uses the feature representations of the training and poisoned data to detect the poisoned data. However, their approach relies on the *strong assumption that the user has access to the poisoned dataset*. Our approach requires access to only the model and the clean training dataset.

Adversarial Training and Backdoor Robustness: Several researchers have studied the relationship between adversarial inputs and poisoned models (including backdoor (Pang et al., 2020; Weng et al., 2020)). Remarkably, Pang et al. (2020) systematically studied the relationship between adversarial inputs and poisoned models in a unified manner by developing a new attack model that jointly optimizes both attacks. This work shows that there is a mutual reinforcement effect between the two attack vectors which can be easily exploited to optimize attacks with respect to multiple metrics. For instance, this work shows that leveraging one attack vector significantly amplifies the effectiveness of the other. Similar to our work (AEGIS), the paper encourages the need to study both attacks by designing countermeasures, albeit from multiple complementary perspectives (e.g., efficacy, fidelity and specificity) to account for the mutual reinforcement effects.

Similarly, researchers have shown that there is a trade-off between adversarial robustness and backdoor attacks. Notably, Weng et al. (2020) demonstrated that adversarial robustness is at odds with backdoor robustness. The authors found that increasing adversarial robustness via adversarial training makes a model more vulnerable to backdoor attacks. Consequently, this trade-off can influence the strength of both attacks and defenses against backdoor attacks. Weng et al. (2020) shows that this trade-off can be leveraged to create more concealed backdoor attacks that evade existing backdoor defenses, and it can also be leveraged to further strengthen some defenses. Similarly, this work (AEGIS) shows that the inherent properties of adversarial training based robust optimization can aid the detection/defense against backdoor attacks in robust models. In the future, we plan to investigate the extent to which increasing or decreasing adversarial robustness may influence the success of backdoor attacks and the defense of AEGIS. In addition, we plan to investigate how to extend AEGIS to achieve joint defense against both adversarial and backdoor attacks.

8. Conclusion

In this paper, we demonstrate a new attack vector for PGD-trained robust DNN models, namely backdoor attacks. We show that such robust models are susceptible to several variants of backdoor attacks, including visible and invisible backdoors. Then, we leverage the inherent properties of these robust ML models to detect this attack. Our proposed detection technique (i.e., AEGIS) is based on clustering the feature representation of PGD-trained robust models to find anomalous clusters. In our evaluation, AEGIS

⁸ <https://github.com/THUYimingLi/backdoor-learning-resources>

Table 14
Standard hyperparameters used for model training.

Dataset	Epochs	LR	Batch Size	LR Schedule
CIFAR-10	110	0.1	128	Drop by 10 at epochs $\in [50, 100]$
MNIST	100	0.1	128	Drop by 10 at epochs $\in [50, 100]$
Fashion-MNIST	100	0.1	128	Drop by 10 at epochs $\in [50, 100]$

Table 15
Backdoor Detection Parameters

Detection Parameters	All Models		
	MNIST	Fashion-MNIST	CIFAR-10
Epsilon (ϵ)	100	100	500
t-SNE Perplexity	30	30	30
Mean shift Bandwidth	35	28	21

Table 16
Detection Efficacy: Number of feature clusters for each class for clean model and visible trigger infected backdoor models

Class Type	Class Labels	MNIST Models			Fashion-MNIST Models			CIFAR-10 Models		
		Backdoor-Infected		Clean	Backdoor-Infected		Clean	Backdoor-Infected		Clean
		Local	Distributed		Local	Distributed		Local	Distributed	
Targeted	{7}	3	3	2	4	3	2	3	4	2
Untargeted	{0 – 6, 8, 9}	2	2	2	2	2	2	2	2	2

accurately detects backdoor-infected PGD-trained robust models and identifies the poisoned class, without any access to the poisoned data, for all visible backdoor triggers. We also found that invisible backdoor triggers are more stealthy and slightly more difficult to detect for AEGIS. Overall, AEGIS detects a backdoor-infected model with 91.6% accuracy (i.e., 11 out of 12 backdoor-infected models), without any false positives. Furthermore, AEGIS detects the targeted class in the backdoor-infected model with a reasonably low (11.1%) false positive rate. Our work reveals that inherent properties of PGD-based robust optimization method allows to expose backdoors. Our code and experimental data are available for replication:

<https://github.com/sakshiudeshi/Expose-Robust-Backdoors>

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to

influence the work reported in this paper

Data Availability

Data will be made available on request.

Acknowledgment

This work was partially funded by OneConnect Financial grant number RGOCFT2001, Singapore Ministry of Education (MOE) President's Graduate Fellowship and the University of Luxembourg's Institute for Advanced Studies (IAS). Ezekiel Soremekun was funded by the IAS Audacity project titled "LAIWYERS: Law and AI: WaYs to Explore Robust Solutions".

Appendix A: Additional Tables and Figures. Additional Tables

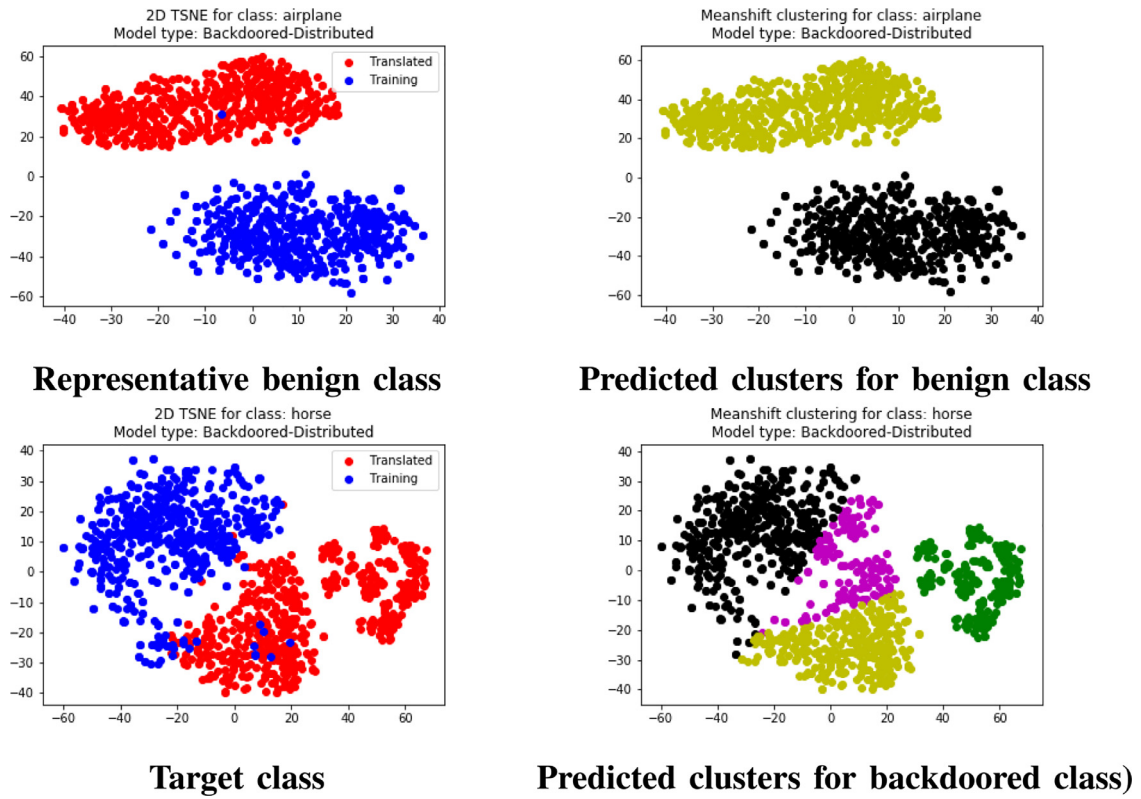


Fig. 14. Feature representation clusters for backdoored CIFAR models (Distributed) with target class *Horse* (7). This figure shows class 0 and 7. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes.

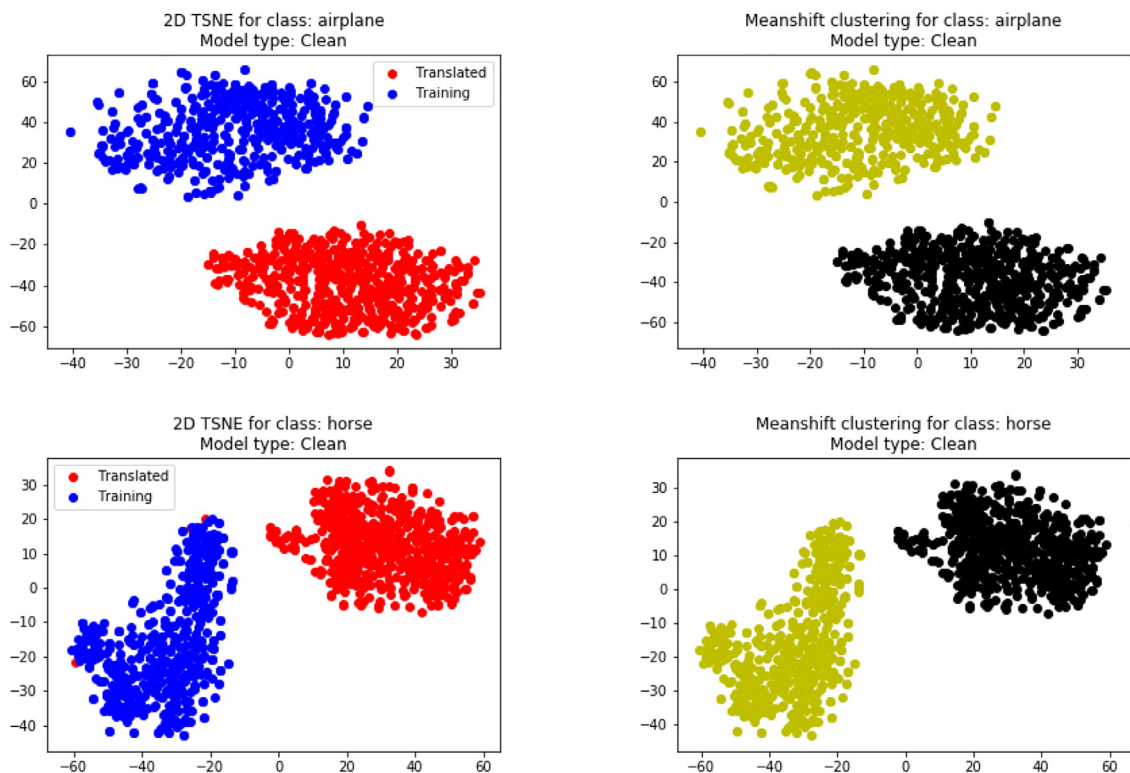


Fig. 15. Feature representation clusters for clean CIFAR10 models. This figure shows class 0 and 7. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes.

Table 17
Detection Efficacy: Number of feature clusters for each class for invisible backdoors

Class Type	Class Labels	MNIST Models		Fashion-MNIST Models		CIFAR-10 Models	
		Backdoor-Infected		Backdoor-Infected		Backdoor-Infected	
		Static	Adversarial	Static	Adversarial	Static	Adversarial
Targeted	{7}	2	2	3	3	3	4
Untargeted	{0}	1	2	3	2	2	2
	{1}	2	2	3	2	2	3
	{2}	2	2	2	2	2	2
	{3}	2	3	2	2	2	2
	{4}	2	2	2	3	2	2
	{5}	2	2	2	2	2	2
	{6}	2	2	2	2	2	2
	{8}	2	2	3	2	2	2
	{9}	2	2	2	2	2	2

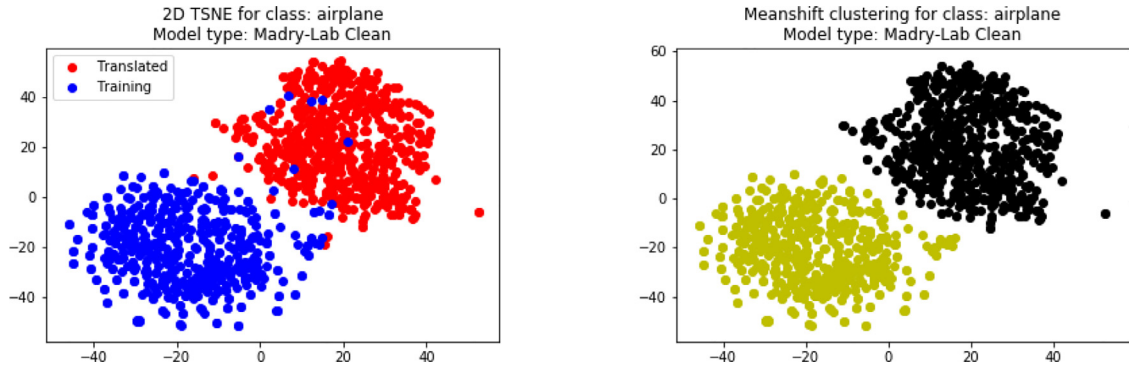


Fig. 16. Feature representation clusters for clean CIFAR10 models from Madry-Lab. This figure shows class 0. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes. It is important to note that the translated images and training set images form separate clusters.

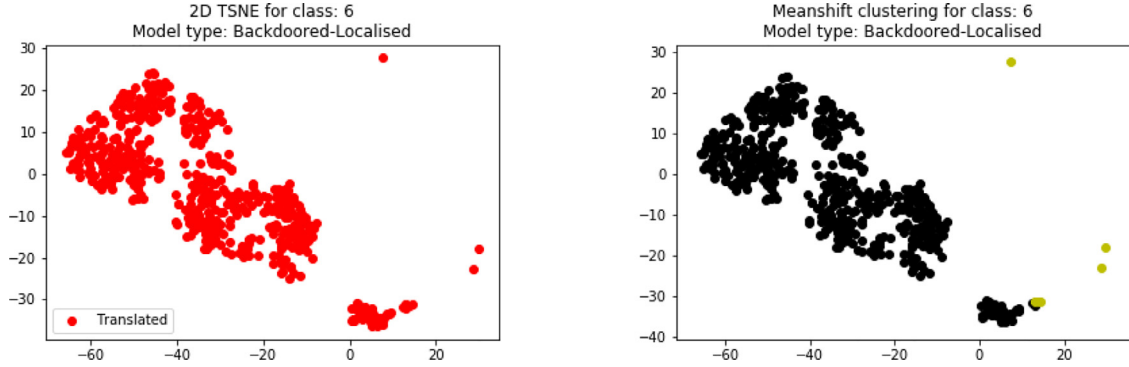


Fig. 17. Representative false positives. These kinds of false positives occur when AEGIS only considers the translated images in the detection for backdoors. This figure shows class 6 of a robust MNIST model poisoned with a localised backdoor. The left column shows the feature representations of the translated and the training images, whereas the right column shows the result of the Mean shift clustering on the corresponding points where different colours represent different classes.

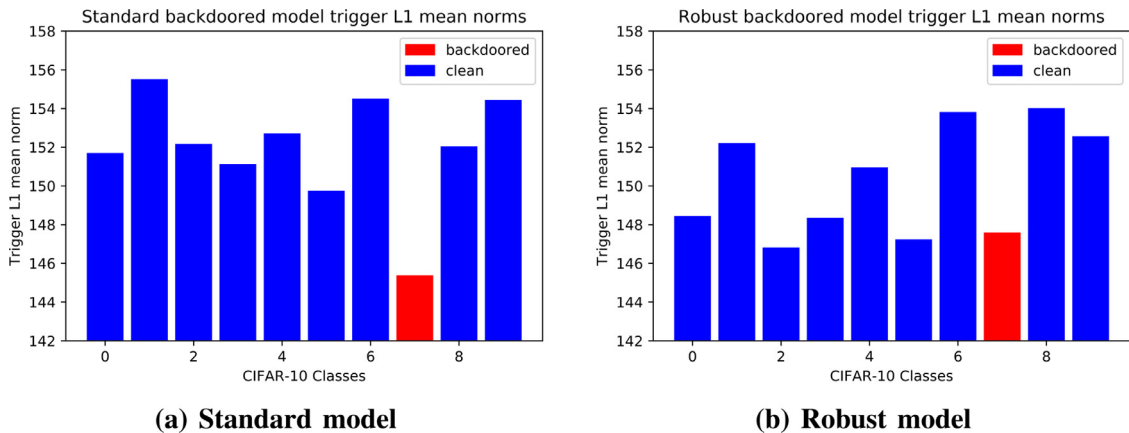


Fig. 18. L1 norms (mean) of the reverse engineered triggers for backdoor-infected standard and robust models. The L1 norms for the reverse engineered triggers are in line with the sizes of the reverse engineered triggers seen in Wang et al. (2019).

References

- Abbasi, M., Gagné, C., 2017. Robustness to adversarial examples through an ensemble of specialists. arXiv preprint arXiv:1702.06856.
- Bagdasaryan, E., Shmatikov, V., 2020. Blind backdoors in deep learning models. arXiv preprint arXiv:2005.03823.
- Chen, B., Carvalho, W., Baracaldo, N., Ludwig, H., Edwards, B., Lee, T., Molloy, I., Srivastava, B., 2019. Detecting backdoor attacks on deep neural networks by activation clustering. In: Workshop on Artificial Intelligence Safety 2019 co-located with the Thirty-Third AAAI Conference on Artificial Intelligence 2019 (AAAI-19), Honolulu, Hawaii, January 27, 2019.
- Chou, E., Tramèr, F., Pellegrino, G., Boneh, D., 2018. Sentinet: Detecting physical attacks against deep learning systems. arXiv preprint arXiv:1812.00292.
- Croce, F., Andriushchenko, M., Sehwag, V., Debenedetti, E., Flammarion, N., Ching, M., Mittal, P., Hein, M., 2020. Robustbench: a standardized adversarial robustness benchmark. arXiv preprint arXiv:2010.09670.
- Croce, F., Hein, M., 2020. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. ICML.
- Croce, F., Hein, M., 2021. Mind the box: l_1 -apgd for sparse adversarial attacks on image classifiers. ICML.
- Doan, B.G., Abbasnejad, E., Ranasinghe, D., 2019. Deepcleanse: A black-box input sanitization framework against backdoor attacks on deepneural networks. arXiv preprint arXiv:1908.03369.
- Du, M., Jia, R., Song, D., 2019. Robust anomaly detection and backdoor attack detection via differential privacy. arXiv preprint arXiv:1911.07116.
- Du, M., Jia, R., Song, D., 2019. Robust anomaly detection and backdoor attack detection via differential privacy. CoRR abs/1911.07116. <http://arxiv.org/abs/1911.07116>
- Dueck, D., 2009. Affinity propagation: clustering data by passing messages. Citeseer.
- Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., 2019. Robustness (python library). <https://github.com/MadryLab/robustness>.
- Fukunaga, K., Hostetler, L.D., 1975. The estimation of the gradient of a density function, with applications in pattern recognition. IEEE Trans. Information Theory 21 (1), 32–40.
- Gao, Y., Xu, C., Wang, D., Chen, S., Ranasinghe, D.C., Nepal, S., 2019. Strip: A defence against trojan attacks on deep neural networks. In: Proceedings of the 35th Annual Computer Security Applications Conference, pp. 113–125.
- Google autolml, 2021. <https://cloud.google.com/autolml>
- Gowda, K.C., Krishna, G., 1978. Agglomerative clustering using the concept of mutual nearest neighbourhood. Pattern recognition 10 (2), 105–112.
- Gu, T., Dolan-Gavitt, B., Garg, S., 2017. Badnets: Identifying vulnerabilities in the machine learning model supply chain. CoRR abs/1708.06733. <http://arxiv.org/abs/1708.06733>
- Guo, C., Wu, R., Weinberger, K.Q., 2020. Trojannet: Embedding hidden trojan horse models in neural networks. arXiv preprint arXiv:2002.10078.
- Guo, W., Wang, L., Xing, X., Du, M., Song, D., 2019. Tabor: A highly accurate approach to inspecting and restoring trojan backdoors in ai systems. arXiv preprint arXiv:1908.01763.
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778.
- He, W., Wei, J., Chen, X., Carlini, N., Song, D., 2017. Adversarial example defense: Ensembles of weak defenses are not strong. 11th USENIX Workshop on Offensive Technologies, WOOT 2017, Vancouver, BC, Canada, August 14–15, 2017.
- Hong, S., Chandrasekaran, V., Kaya, Y., Dumitras, T., Papernot, N., 2020. On the effectiveness of mitigating data poisoning attacks with gradient shaping. arXiv preprint arXiv:2002.11497.
- Huang, X., Alzantot, M., Srivastava, M., 2019. Neuroninspect: Detecting backdoors in neural networks via output explanations. arXiv preprint arXiv:1911.07399.
- Ilyas, A., Santurkar, S., Tsipras, D., Engstrom, L., Tran, B., Madry, A., 2019. Adversarial examples are not bugs, they are features. Advances in neural information processing systems 32.
- Isola, P., Zhu, J.-Y., Zhou, T., Efros, A.A., 2017. Image-to-image translation with conditional adversarial networks. In: The IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Kariyappa, S., Qureshi, M.K., 2019. Improving adversarial robustness of ensembles with diversity training. arXiv preprint arXiv:1901.09981.
- Kaur, S., Cohen, J., Lipton, Z.C., 2019. Are perceptually-aligned gradients a general property of robust classifiers? CoRR abs/1910.08640. <http://arxiv.org/abs/1910.08640>
- Krizhevsky, A., Hinton, G., et al., 2009. Learning multiple layers of features from tiny images.
- LeCun, Y., Cortes, C., Burges, C.J., 1998. The mnist database of handwritten digits, 1998. URL <http://yann.lecun.com/exdb/mnist> 10, 34.
- Li, R., Zhang, H., Yang, P., Huang, C.-C., Zhou, A., Xue, B., Zhang, L., 2021. Ensemble defense with data diversity: Weak correlation implies strong robustness. arXiv preprint arXiv:2106.02867.
- Li, Y., Wu, B., Jiang, Y., Li, Z., Xia, S.-T., 2020. Backdoor learning: A survey. arXiv preprint arXiv:2007.08745.
- Likas, A., Vlassis, N., Verbeek, J.J., 2003. The global k-means clustering algorithm. Pattern recognition 36 (2), 451–461.
- Liu, K., Dolan-Gavitt, B., Garg, S., 2018. Fine-pruning: Defending against backdoor-ing attacks on deep neural networks. In: Research in Attacks, Intrusions, and Defenses - 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10–12, 2018, Proceedings, pp. 273–294.
- Liu, M.-Y., Breuel, T., Kautz, J., 2017. Unsupervised image-to-image translation networks. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), Advances in Neural Information Processing Systems 30. Curran Associates, Inc., pp. 700–708. <http://papers.nips.cc/paper/6672-unsupervised-image-to-image-translation-networks.pdf>
- Liu, Y., Lee, W.-C., Tao, G., Ma, S., Aafer, Y., Zhang, X., 2019. Abs: Scanning neural networks for back-doors by artificial brain stimulation. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, pp. 1265–1282.
- Liu, Y., Ma, S., Aafer, Y., Lee, W.-C., Zhai, J., Wang, W., Zhang, X., 2018. Trojaning attack on neural networks. 25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18–22, 2018. The Internet Society.
- Maaten, L.v.d., Hinton, G., 2008. Visualizing data using t-sne. Journal of machine learning research 9 (Nov), 2579–2605.
- MacKiewicz, A., Ratajczak, W., 1993. Principal components analysis (pca). Computers & Geosciences 19 (3), 303–342.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A., 2018. Towards deep learning models resistant to adversarial attacks. In: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 – May 3, 2018, Conference Track Proceedings.
- McInnes, L., Healy, J., Astels, S., 2017. hdbscan: Hierarchical density based clustering. J. Open Source Softw. 2 (11), 205.
- McInnes, L., Healy, J., Saul, N., Großberger, L., 2018. Umap: Uniform manifold approximation and projection. Journal of Open Source Software 3 (29), 861.
- Meng, Y., Su, J., O’Kane, J., Jamshidi, P., 2020. Athena: A framework based on diverse weak defenses for building adversarial defense. arXiv preprint arXiv:2001.00308.
- Microsoft azure cognitive services, 2021. <https://azure.microsoft.com/en-us/services/cognitive-services/>
- Ng, A., Jordan, M., Weiss, Y., 2001. On spectral clustering: Analysis and an algorithm. Advances in neural information processing systems 14.
- Pang, R., Shen, H., Zhang, X., Ji, S., Vorobeychik, Y., Luo, X., Liu, A., Wang, T., 2020. A tale of evil twins: Adversarial inputs versus poisoned models. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, pp. 85–99.
- Pang, T., Xu, K., Du, C., Chen, N., Zhu, J., 2019. Improving adversarial robustness via promoting ensemble diversity. In: International Conference on Machine Learning. PMLR, pp. 4970–4979.
- Papernot, N., McDaniel, P.D., Goodfellow, I.J., Jha, S., Celik, Z.B., Swami, A., 2017. Practical black-box attacks against machine learning. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, AsiaCCS 2017, Abu Dhabi, United Arab Emirates, April 2–6, 2017, pp. 506–519.
- Papernot, N., McDaniel, P.D., Jha, S., Fredrikson, M., Celik, Z.B., Swami, A., 2016. The limitations of deep learning in adversarial settings. In: IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21–24, 2016, pp. 372–387.
- Papernot, N., McDaniel, P.D., Wu, X., Jha, S., Swami, A., 2016. Distillation as a defense to adversarial perturbations against deep neural networks. In: IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22–26, 2016, pp. 582–597.
- Qiao, X., Yang, Y., Li, H., 2019. Defending neural backdoors via generative distribution modeling. In: Advances in Neural Information Processing Systems, pp. 14004–14013.
- Raghunathan, A., Steinhardt, J., Liang, P., 2018. Certified defenses against adversarial examples. arXiv preprint arXiv:1801.09344.
- Raghunathan, A., Steinhardt, J., Liang, P., 2018. Certified defenses against adversarial examples. In: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 – May 3, 2018, Conference Track Proceedings. <https://openreview.net/forum?id=Bys40b-Rb>
- Santurkar, S., Ilyas, A., Tsipras, D., Engstrom, L., Tran, B., Madry, A., 2019. Image synthesis with a single (robust) classifier. In: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, 8–14 December 2019, Vancouver, BC, Canada, pp. 1260–1271.
- Sinha, A., Namkoong, H., Duchi, J., 2017. Certifying some distributional robustness with principled adversarial training. arXiv preprint arXiv:1710.10571.
- Sinha, A., Namkoong, H., Duchi, J.C., 2018. Certifying some distributional robustness with principled adversarial training. In: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 – May 3, 2018, Conference Track Proceedings. <https://openreview.net/forum?id=Hk6kPgZA-Szegedy>
- Szegedy, C., Zarembka, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I.J., Fergus, R., 2014. Intriguing properties of neural networks. In: 2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14–16, 2014, Conference Track Proceedings.
- Tang, D., Wang, X., Tang, H., Zhang, K., 2019. Demon in the variant: Statistical analysis of dnns for robust backdoor contamination detection. arXiv preprint arXiv:1908.00686.
- Tran, B., Li, J., Madry, A., 2018. Spectral signatures in backdoor attacks. In: Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, 3–8 December 2018, Montréal, Canada, pp. 8011–8021.
- Tsipras, D., Santurkar, S., Engstrom, L., Turner, A., Madry, A., 2019. Robustness may be at odds with accuracy. In: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019.
- Udeshi, S., Peng, S., Woo, G., Loh, L., Rawshan, L., Chattopadhyay, S., 2019. Model agnostic defence against backdoor attacks in machine learning. CoRR abs/1908.02203. <http://arxiv.org/abs/1908.02203>

- Veldanda, A.K., Liu, K., Tan, B., Krishnamurthy, P., Khorrami, F., Karri, R., Dolan-Gavitt, B., Garg, S., 2020. Nnuculation: Broad spectrum and targeted treatment of backdoored dnns. *arXiv preprint arXiv:2002.08313*.
- Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., Zhao, B.Y., 2019. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In: 2019 IEEE Symposium on Security and Privacy, SP 2019, Proceedings, 20-22 May 2019, San Francisco, California, USA.
- Weng, C.-H., Lee, Y.-T., Wu, S.-H.B., 2020. On the trade-off between adversarial and backdoor robustness. *Advances in Neural Information Processing Systems* 33, 11973–11983.
- Wong, E., Kolter, J.Z., 2018. Provable defenses against adversarial examples via the convex outer adversarial polytope. In: *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmssmässan, Stockholm, Sweden, July 10-15, 2018*, pp. 5283–5292. <http://proceedings.mlr.press/v80/wong18a.html>
- Wong, E., Kolter, Z., 2018. Provable defenses against adversarial examples via the convex outer adversarial polytope. In: *International Conference on Machine Learning*, pp. 5286–5295.
- Wong, E., Schmidt, F., Metzen, J.H., Kolter, J.Z., 2018. Scaling provable adversarial defenses. In: *Advances in Neural Information Processing Systems*, pp. 8400–8409.
- Xiang, Z., Miller, D.J., Kesidis, G., 2020. Detection of backdoors in trained classifiers without access to the training set. *IEEE Transactions on Neural Networks and Learning Systems*.
- Xiao, H., Rasul, K., Vollgraf, R., 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *cs.LG/1708.07747*.
- Xu, X., Wang, Q., Li, H., Borisov, N., Gunter, C.A., Li, B., 2019. Detecting ai trojans using meta neural analysis. *arXiv preprint arXiv:1910.03137*.
- Yao, Y., Li, H., Zheng, H., Zhao, B.Y., 2019. Latent backdoor attacks on deep neural networks. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, pp. 2041–2055. doi:10.1145/3319535.3354209.
- Yi, Z., Zhang, H., Tan, P., Gong, M., 2017. Dualgan: Unsupervised dual learning for image-to-image translation. In: *The IEEE International Conference on Computer Vision (ICCV)*.
- Zhong, H., Liao, C., Squicciarini, A.C., Zhu, S., Miller, D., 2020. Backdoor embedding in convolutional neural network models via invisible perturbation. In: *Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy*, pp. 97–108.
- Zhu, J.-Y., Park, T., Isola, P., Efros, A.A., 2017. Unpaired image-to-image translation using cycle-consistent adversarial networks. In: *The IEEE International Conference on Computer Vision (ICCV)*.
- Zou, M., Shi, Y., Wang, C., Li, F., Song, W., Wang, Y., 2018. Potrojan: powerful neural-level trojan designs in deep learning models. *arXiv preprint arXiv:1802.03043*.

Ezekiel Soremekun is a Research Scientist at SnT, University of Luxembourg. Ezekiel's research is focused on the automated software validation including automated debugging, software testing and analysis.

Sakshi Udeshi is a PhD student at the Singapore University of Technology and Design. His research focus is broadly software infrastructure for machine learning which includes, but is not limited to testing, fairness, and security amongst others.

Sudipta Chattopadhyay received the Ph.D. degree in computer science from the National University of Singapore, Singapore, in 2013. He is an Assistant Professor with the Information Systems Technology and Design Pillar, Singapore University of Technology and Design, Singapore. In his doctoral dissertation, he researched on Execution-Time Predictability, focusing on Multicore Platforms. He seeks to understand the influence of execution platform on critical software properties, such as performance, energy, robustness, and security. His research interests include program analysis, embedded systems, and compilers. Mr. Chattopadhyay serves in the review board of the IEEE Transactions on Software Engineering